

LEXA: AN AI VOICE CONVERSATION AND SPEECH COACH

PROJECT REPORT [Phase-I/II/III]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Information Technology

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)

Submitted by

Name of the student1 (Scholar number and Semester)
Name of the student2 (Scholar number and Semester)

Under the supervision of

Name of the supervisor
Designation of the supervisor
(Name of Department)

Month & Year

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled “PROJECT TITLE” has been satisfactorily completed by STUDENT NAME (SCHOLAR NO.) It is a bonafide piece of work, carried out under our guidance in the Department of Information Technology, Indian Institute of Information Technology, Bhopal for the partial fulfillment of the Bachelor of Engineering during the academic year 202_-__.

Date:

Head/Department Coordinator,
Department of IT,
IIIT Bhopal (M.P.)

Name of Supervisor(s),
Designation,
Department,
IIIT Bhopal (M.P.)

DECLARATION

We hereby declare that the following major project synopsis entitled “PROJECT TITLE” presented in the is the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology. It is an authentic documentation of our original work carried out under the able guidance of Name of Supervisors. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Name of the student1 (Scholar number) Sign of student1

Name of the student2 (Scholar number) Sign of student2

ACKNOWLEDGEMENT

We, [Student 1 Name] and [Student 2 Name], would like to express our sincere gratitude to everyone who supported and guided us throughout the successful completion of our minor project titled "[Project Title]".

First and foremost, we would like to extend our heartfelt thanks to our esteemed Director, [Director's Name], for providing us with the necessary resources and facilities for conducting our project. His/her encouragement and vision played a key role in helping us accomplish our goals.

We are deeply grateful to our Project Guide, [Guide's Name], for his/her constant guidance, invaluable suggestions, and expert advice throughout the project. His/her knowledge and insights helped us enhance the quality of our work, and his/her support kept us motivated during difficult times.

Our sincere thanks to the Department Coordinator, [Coordinator's Name], for the valuable support and encouragement provided during completion of the project.

We would also like to express our gratitude to all the faculties of the [Department Name] for their valuable guidance and support in clarifying doubts and providing us with the right direction throughout the smooth execution of our project.

A special thanks to our friends for their continuous support, valuable suggestions, and helping us during various stages of the project. Their insights and encouragement motivated us to keep working on the project with enthusiasm.

Last but certainly not the least, we would like to express our deepest gratitude to our parents and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind our efforts.

We are truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

[Student 1 Name]

[Student 2 Name]

[Roll Numbers]

AREA OF WORK

The work reported in this project lies in the area of applied speech and language engineering, at the intersection of automatic speech recognition, natural language processing, phonetics and web application development. Its subject is computer-assisted language learning: the use of speech and language models not to replace a teacher, but to give a learner a structured account of their own spoken performance.

On the speech side, the project covers the acquisition and normalisation of audio, voice-activity detection in the browser, routing between speech-recognition providers according to the language being spoken, and phoneme-level acoustic analysis using a self-supervised speech model. On the language side, it covers conversational response generation, structured extraction of grammatical and lexical feedback, and the validation of that feedback against the learner's transcript. The language-bridge component extends the work to multilingual and code-mixed settings, where the learner's language, its script and its English loanwords all have to be modelled explicitly.

On the software side, the project is a full-stack web system. It includes a typed HTTP API, asynchronous database persistence, replaceable provider adapters with graceful fallback, deterministic scoring, an offline test suite, a React front end with server-side rendering, and a containerised deployment on managed infrastructure. The engineering emphasis is on dependability and explainability: every score must be traceable to evidence, and every optional component must degrade without breaking a session.

The project therefore belongs to the broader area of intelligent tutoring and speech-enabled interfaces, with a specific contribution in making multi-dimensional spoken-language feedback practical on modest, commodity infrastructure.

TABLE OF CONTENT

S. No.	Title	Page No.
—	Certificate	i
—	Declaration	ii
—	Acknowledgement	iii
—	Area of Work	iv
—	List of Figures	vi
—	List of Tables	vii
—	Abstract	1
1	Introduction	2
2	Literature Review	5
3	Problem Definition and Objectives	10
4	Proposed Methodology and Work Description	13
5	Proposed Algorithms	19
6	Proposed Flowchart, Block Diagram and Data-Flow Diagrams	24
7	Implementation: Tools, Technology and Code	28
8	Result Discussion and Analysis	35
9	Conclusion and Future Scope	41
10	References	43

LIST OF FIGURES

Fig. No.	Description	Page No.
1	Layered architecture of the system	14
2	Sequence of messages for one conversational turn	15
3	Pronunciation analysis pipeline: acoustic and reference paths, alignment and evidence filters	21
4	Level-0 data-flow diagram (context diagram)	24
5	Level-1 data-flow diagram	25
6	Video review workflow	26
7	Language-bridge workflow, including deterministic validation of alternatives	27
8	Entity-relationship model of the persistence layer	30
9	Front-end route flow and report generation	31
10	Deployment topology	33

LIST OF TABLES

Table No.	Description	Page No.
1	Session modes and their outputs	11
2	Supported spoken languages and speech-recognition routing	11
3	Principal analysis thresholds and limits	22
4	Principal technologies and their roles	28
5	API surface	29
6	Selected deployment configuration	34
7	Automated test coverage by behaviour	35
8	Observed latency for one turn (milliseconds)	36
9	Fluency analysis on two generated samples	36
10	Pronunciation analysis on correct and deliberately mispronounced speech	37
11	Time to complete audio for one sentence	37
12	Qualitative comparison of feedback approaches	39

ABSTRACT

Speaking a second language is a composite skill, and a learner who wants to improve it needs feedback on several different things at once: whether a sentence was grammatical, whether the words were varied and natural, whether the delivery was fluent, and whether individual sounds were articulated clearly. Human coaching supplies this feedback, but it is expensive, episodic and difficult to obtain at the frequency that sustained practice requires. Existing language applications usually address one of these dimensions in isolation, and most of them are designed for a single language and a single script.

This report describes Lexa, a web-based voice conversation and speech-coaching system that was designed, implemented and deployed as a B.Tech project. A learner holds a spoken conversation with an artificial-intelligence partner in the browser, and when the conversation ends the system produces a report on four dimensions of the learner's speech: grammar, vocabulary, fluency and phoneme-level pronunciation. The same analyses can be applied to an uploaded English video of up to five minutes, so that a recording made elsewhere can be reviewed without a live call. A separate language-bridge mode allows a learner to speak Hindi, Kannada, Tamil or Telugu, be answered in simple English, and receive a list of English equivalents for the words and phrases actually used.

The system is implemented as a FastAPI backend and a React 19 / TanStack Start frontend. Speech recognition, language modelling and speech synthesis are reached through replaceable provider adapters; the deployed configuration uses Groq for Whisper transcription, an open-weight GPT-OSS model for dialogue and grammar/vocabulary suggestions, Orpheus for synthesis, Sarvam saaras for the three Dravidian languages and Deepgram as a speech-synthesis fallback. Pronunciation analysis is the only component that runs a model locally: a 316-million-parameter wav2vec2 CTC phoneme recogniser, aligned against dictionary-derived phonemes with the Needleman-Wunsch algorithm and filtered so that only recurring, confident patterns are reported. Scores for grammar, vocabulary, fluency and pronunciation are computed in application code rather than requested from a language model, which makes them deterministic and explainable.

The implementation is accompanied by an offline automated test suite of thirty modules, with a recorded run of 454 passing tests. Measured single-turn latency was 1,968 ms for an English turn and 7,599 ms for a Kannada bridge turn, dominated in the second case by the additional speech-recognition and synthesis calls. On generated evaluation audio, the fluency analyser separated a clean sample (score 100, 158.9 words per minute) from a hesitant one (score 19, 83.9 words per minute), and the pronunciation analyser reduced eighteen raw phoneme mismatches to one trustworthy finding while reporting none for correctly articulated speech. The report discusses these results, the design decisions behind them and the limitations that remain, including the absence of a controlled word-error-rate benchmark and the constraints imposed by a free-tier deployment.

1. INTRODUCTION

1.1 Background and Motivation

English is the working language of a large part of Indian higher education and professional life, yet most learners acquire it through reading and writing rather than through sustained speaking practice. The gap is not primarily one of vocabulary knowledge. A learner may know the rule for the past tense and still say “I go to college yesterday” under the cognitive load of real conversation; may know the word “improve” and still repeat “very good” four times in a minute; may know how “think” is spelled and still pronounce it with a /t/ rather than a /θ/. These are performance problems, and performance problems are corrected by practice followed by specific, timely feedback.

The feedback loop is the expensive part. A human coach can hear all of these problems at once and can decide which of them matter for a particular learner, but coaching is constrained by cost, scheduling and the availability of trained teachers. Self-study applications remove the scheduling constraint but usually narrow the feedback: a grammar quiz checks grammar, a pronunciation drill checks a fixed inventory of sounds, and a conversational chatbot provides practice without assessment. What is missing is a single artefact that lets a learner speak freely, then shows what was said, how it was said, and what to work on next.

The Common European Framework of Reference for Languages describes spoken proficiency in terms of communicative performance rather than isolated knowledge [1]. That framing motivates the design of the present system: the learner should be assessed while speaking, on multiple dimensions, and the assessment should be tied to what the learner actually produced rather than to a pre-authored exercise. Recent progress in automatic speech recognition, open-weight language models and self-supervised speech representations makes such an assessment feasible on commodity infrastructure, provided that the system is engineered so that model output is checked and scored by deterministic code.

1.2 The System in Brief

Lexa is a browser application with two entry points. In the first, the learner starts a live conversation with an artificial-intelligence partner. Speech is captured in the browser, transmitted a turn at a time to a backend service, transcribed, answered by a language model, and returned as synthesized speech, so that the interaction is spoken at both ends. The conversation is stored as a session, and when the learner ends it, the report page runs four analyses over the learner's own turns and presents scores, findings and short practice suggestions.

The four analyses are complementary rather than redundant. Grammar analysis identifies errors in tense, articles, prepositions, agreement, plurals, word form and sentence structure, quoting the sentence and offering a correction. Vocabulary analysis reports repeated words and phrases, over-used basic words and expressions that are grammatical but unnatural, with better alternatives. Fluency analysis measures speaking rate, pauses, filler words, immediate repetitions and false starts from the word timings captured during transcription; it deliberately

does not ask a language model for an opinion about fluency. Pronunciation analysis compares the sounds that were produced against the sounds the transcribed words require, at phoneme level, and reports only patterns that recur across different words.

The second entry point accepts an uploaded English video of up to five minutes. The backend validates the file, extracts and normalises its audio track, transcribes it, and stores the result as a completed session containing a single learner turn. The report page then runs the same grammar, vocabulary and fluency analyses over that turn. This path exists because much speaking practice happens away from a microphone attached to a web application, and because a recorded presentation or classroom interaction is a natural object of review.

A third mode, the language bridge, serves learners who are not yet ready to hold a whole conversation in English. The learner speaks Hindi, Kannada, Tamil or Telugu; the partner replies in simple English, which the learner can hear and read; and at the end the system reports English equivalents for the native-language words and phrases that were actually used. The bridge is deliberately unscored, because judging a Hindi utterance with English grammar rules would produce a number that means nothing. English coaching remains the core product, and the bridge is an adjacent feature that reuses the same conversation, persistence and reporting machinery.

1.3 Users, Modes and Languages

The primary user is an individual learner of English, working alone, on a laptop or desktop browser with a microphone. The system keeps no accounts and no personal profile; a session is identified by an opaque identifier, and the learner reaches the report through that identifier. Three session modes exist. English Coach is the core mode: the learner speaks English and is coached on how they said it. Language Bridge is the mode described above. Video Coach is a finished session built from an uploaded video rather than a live call; it holds one learner turn and no assistant reply, and the English analyses run over it exactly as they do for a conversation.

Four spoken languages are supported in the bridge, each with its own script and its own lexical data: Hindi (Devanagari), Kannada (Kannada script), Tamil (Tamil script) and Telugu (Telugu script). English is the coached language and is never offered as a bridge language. The interface exposes the languages before any backend request is made, so that the landing page can present the choice without a network round trip.

1.4 Scope and Boundaries

The project covers the complete path from browser capture to a persisted report: audio acquisition and voice-activity detection in the browser, upload and normalisation, transcription, dialogue, synthesis, session persistence, four analyses, two alternative report types, and a deployment configuration on managed infrastructure. It is a working system rather than a prototype of a single component: a session can be created, held, ended and reviewed, and the same deployment serves both the application programming interface and the front end.

Several boundaries were drawn deliberately. There is no authentication or user account; the API is rate limited per client address instead. Interaction is turn-based over HTTPS rather than streamed over a persistent socket, so the learner waits for a complete reply rather than hearing

it as it is generated. Cross-session progress tracking and a unified report that deduplicates findings across analyses were planned as later phases and are not part of this submission. On the free deployment, pronunciation analysis is disabled because the local phoneme model requires more memory than the instance provides. These are stated as scope decisions, not as oversights, and each is revisited in the discussion of limitations and future work.

1.5 Organisation of the Report

Chapter 2 reviews the literature that informed the design: spoken-language assessment, self-supervised and weakly supervised speech recognition, transformer language models, pronunciation assessment, fluency research, code-mixing and browser speech capture. Chapter 3 states the problem, the objectives and the scope in precise terms. Chapter 4 describes the methodology and the architecture, including the lifecycle of a conversational turn. Chapter 5 presents the algorithms and the scoring rules. Chapter 6 presents the data-flow diagrams, block diagrams and workflows. Chapter 7 documents the implementation, the technology stack, the database, the front end, the test suite and the deployment. Chapter 8 reports and discusses the results, including measured latencies, evaluation samples, limitations and threats to validity. Chapter 9 concludes and sets out future work.

2. LITERATURE REVIEW

The system sits at the intersection of four research traditions: the assessment of spoken language proficiency, automatic speech recognition, large language models and pronunciation assessment. This chapter surveys each in turn, together with the smaller bodies of work on fluency measurement, code-mixed speech and browser-based speech capture, and closes by identifying the gaps that the present implementation addresses.

2.1 Assessing Spoken Language Proficiency

The dominant framework for describing language proficiency is the Common European Framework of Reference for Languages, which defines six levels from A1 to C2 in terms of what a learner can do in reception, production, interaction and mediation [1]. Its descriptors are communicative: a B1 speaker, for example, can sustain a conversation on familiar matters and describe experiences and ambitions, even though errors remain. The framework is explicitly not a marking scheme for individual utterances, and it does not prescribe how to measure the constructs it describes. That gap between a descriptive framework and an operational measurement is where automated assessment has to make design choices.

Human assessment of spontaneous speech typically decomposes into analytic dimensions: grammatical accuracy, lexical range, fluency, pronunciation and coherence. Analytic scoring has the advantage of diagnostic value, because a learner can be told which dimension is weak, but it multiplies the cost of human judgement and introduces rater variability. The practical consequence for an automated system is that it should produce dimension-level evidence rather than a single holistic number, and it should keep the evidence traceable to what the learner said. Lexa adopts this analytic decomposition directly: four dimensions, each with its own findings, and no attempt to collapse them into one proficiency level.

2.2 Self-Supervised Speech Representation Learning

Speech recognition moved, over the past decade, from pipelines of hand-engineered features and hidden Markov models to end-to-end neural systems trained on large corpora. A pivotal step was self-supervised pre-training. wav2vec 2.0 learns representations from unlabelled audio by masking latent speech representations and solving a contrastive task, and then fine-tunes with a connectionist temporal classification objective on a comparatively small amount of labelled data [2]. The approach reduced the labelled data required for competitive recognition and made it practical to fine-tune a single architecture for many languages.

Cross-lingual pre-training extended the idea to languages with little labelled data. XLSR learns shared representations across dozens of languages from unlabelled speech and then fine-tunes per language, showing that multilingual pre-training transfers useful acoustic structure even when the target language was not the focus of pre-training [3]. Public multilingual corpora such as Common Voice, which collects validated recordings from volunteers in many languages, made such pre-training and evaluation possible at scale [4]. For a project concerned with Indian languages, these results matter twice: they explain why a general multilingual recogniser can be adequate on high-resource languages and weak on low-resource ones, and they make a

phoneme-level acoustic model feasible on ordinary hardware, because a fine-tuned wav2vec2 model is small enough to run on a CPU.

The pronunciation component of Lexa uses a wav2vec2 model fine-tuned to emit phonemes rather than characters. The model is loaded locally, performs a frame-level classification every 20 milliseconds, and its output is collapsed with the standard CTC rule. This design follows directly from the self-supervised line of work: the acoustic representation is learned, the phoneme inventory is fixed by the fine-tuning data, and the decoder is a small amount of deterministic code.

2.3 Weakly Supervised Recognition at Scale

Whisper took a different route. Instead of self-supervision followed by fine-tuning, it trained a sequence-to-sequence transformer on 680,000 hours of weakly supervised audio collected from the web, with multilingual transcription, translation and language identification as joint objectives [5]. The result is a family of models with strong multilingual recognition, robustness to accents and background noise, and word-level timestamps. Whisper also demonstrated that scale and data diversity can substitute, to a degree, for careful in-domain curation.

For a deployed application, Whisper's practical attractions are that it is available behind hosted, OpenAI-compatible APIs and that it returns word-level timings on request. Those timings are the input to any measurement of speaking rate and pausing, which is why fluency analysis in Lexa depends on them. Its limitation is equally relevant: performance is uneven across languages, and the long tail of the training distribution is thin. The project observed this directly, with auto-detection labelling Kannada as Tamil and Telugu as Hindi, and with more spelling noise in the three Dravidian languages than in Hindi. The engineering response was to pin the language on every request and to route the three Dravidian languages to an Indic-first recogniser, a decision discussed in Chapter 4.

2.4 Transformer Language Models

The transformer architecture replaced recurrence with self-attention, allowing sequence models to be trained in parallel and to capture long-range dependencies [6]. Language modelling then scaled: generative pre-training followed by task-specific fine-tuning produced strong transfer [7], and further scaling showed that a sufficiently large model could perform new tasks from a handful of examples in its prompt, without parameter updates [8]. This few-shot behaviour is the basis of the conversational partner in Lexa: the system prompt defines the role, the constraints and the reply style, and the session history is replayed as context on every turn.

Open-weight models have made this capability available outside a single vendor. The GPT-OSS family, released with published model cards, provides reasoning-capable transformer models that can be served by third-party inference providers [9]. Lexa uses a 120-billion-parameter member of this family through Groq's inference service, with a low reasoning effort setting and a bounded output length, because a conversational reply should be short and should arrive quickly. Two engineering lessons from the literature are reflected in the implementation. First, structured output is best requested explicitly and then validated: the grammar and vocabulary adapters request JSON and still check every field. Second, a language model is a

proposal engine, not a measuring instrument. Counting, scoring and thresholding are therefore performed in code, a principle that runs through Chapter 5.

2.5 Pronunciation Assessment and Phoneme Alignment

Pronunciation assessment compares what a speaker produced with what the target words require. Classical approaches estimate a goodness-of-pronunciation score from the posterior probability of the expected phonemes given the acoustic model, which requires a phoneme recogniser aligned to the expected sequence. More recent work replaces the hand-built pronunciation lexicon and forced alignment with a CTC phoneme recogniser whose output is compared to a dictionary pronunciation, which is simpler to deploy but produces insertion and deletion noise at the sequence level.

Comparing two phoneme sequences position by position is fragile. A single dropped or inserted sound shifts every subsequent position, so a learner who omits one consonant appears to mispronounce the rest of the word. The standard remedy is global sequence alignment. The Needleman-Wunsch algorithm, originally formulated for comparing amino-acid sequences, computes an optimal alignment under a substitution cost and a gap penalty using dynamic programming [10]. Applied to phonemes, it tolerates insertions and deletions while still identifying genuine substitutions. Lexa uses this algorithm, with unit substitution and gap costs, between the expected phoneme sequence and the recognised one.

The expected sequence comes from a pronouncing dictionary. The CMU Pronouncing Dictionary provides ARPAbet transcriptions for a large English vocabulary [11], and the g2p_en package extends coverage to words not in the dictionary through a neural grapheme-to-phoneme model [12]. The recognised sequence comes from the acoustic model in International Phonetic Alphabet (IPA) symbols, so the two alphabets must be mapped onto a common inventory before alignment. This notation gap, and the treatment of reduced vowels in function words, are the main sources of false positives in phoneme-level feedback, and they are addressed explicitly in the filtering stage described in Chapter 5.

2.6 Fluency, Pauses and Disfluencies

Fluency is not the absence of hesitation. Spontaneous speech contains filled pauses, repetitions and repairs, and psycholinguistic research has shown that “uh” and “um” are not random noise but signals used by speakers to manage planning and turn-taking [13]. An assessment system therefore has to distinguish habitual disfluency from the ordinary management of spontaneous speech, and it has to measure rate and pausing in a way that is not distorted by the interlocutor.

Two measurement decisions follow. First, pauses must be measured within a speaker's own turns, because silence between turns belongs to the other speaker; summing gaps across a dialogue would attribute the partner's thinking time to the learner. Second, speaking rate and articulation rate should be distinguished: the former includes pause time, the latter excludes it, and the difference between them is itself diagnostic. Lexa implements both, together with a comfortable band for rate, and counts filled pauses, immediate word repetitions and restarts as separate finding types so that the report can say which habit is present rather than merely that fluency is poor.

2.7 Code-Mixing, Scripts and Indic Languages

Everyday speech in Hindi, Kannada, Tamil and Telugu routinely mixes English words into native-language sentences. Recognition may render such words in the native script or in Latin letters, and the choice is not stable across engines or utterances. For a vocabulary report this matters: an English word that the learner already knows should not be presented as new vocabulary, whereas a native word that the learner does not know in English should be. Distinguishing the two requires lexical knowledge, not merely a language identifier, and it requires script-aware validation.

Scripts are formally defined by the Unicode Standard, which assigns each script a block of code points and specifies its character properties [14]. That gives a reliable, language-independent test for whether a string is written in Devanagari, Kannada, Tamil or Telugu, and whether it contains Latin letters. Unicode properties were used in this project as the first line of defence against a class of error in which a language model “translates” an English word the learner already used. The second line is a curated list of common English loanwords in each script, since a loanword written in Devanagari is still an English word.

2.8 Browser Speech Capture and Voice Activity Detection

The browser has become a viable speech client. The MediaRecorder interface exposes compressed audio encoding from a microphone stream and is supported, with different container preferences, across major browsers [15]. Capture alone is not enough for a conversation: the client must decide when the learner has finished speaking. Voice activity detection can be performed on the server, but doing it in the browser avoids uploading silence and allows the interface to respond immediately. Silero VAD is a small, openly available neural voice-activity detector that runs through ONNX Runtime and is accurate enough for interactive use [16].

Lexa runs Silero VAD in the browser through a WebAssembly build of ONNX Runtime. The model and runtime are several megabytes, so they are loaded from a content-delivery network rather than bundled with the application, and manual recording remains available if the detector fails. The design choice is consistent with the literature: detection close to the audio source reduces bandwidth and latency, while the server still validates and normalises every upload.

2.9 Summary of Gaps Addressed

The literature supplies strong components but few complete systems that combine them for a learner-facing product. Five gaps are relevant to this project. First, assessment research tends to evaluate a single dimension, whereas a learner needs several dimensions reported together against the same conversation. Second, language-model-based feedback is usually accepted as generated, without deterministic validation of counts, quotations or scores. Third, phoneme-level pronunciation feedback is known to produce false positives on accented speech, and published systems seldom describe the filters that make it usable. Fourth, Indic-language support is usually studied as recognition accuracy, not as a learner-facing bridge from the learner's language to English. Fifth, deployment constraints, such as running a speech pipeline on a free-tier instance, are rarely treated as part of the design.

Lexa addresses these gaps by treating the conversation as the unit of assessment, by separating model judgement from deterministic scoring and validation, by filtering pronunciation evidence before it reaches the learner, by implementing a validated native-language bridge, and by documenting a deployment in which the expensive local model is optional. The following chapter states the resulting problem and objectives precisely.

3. PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem Statement

A learner of spoken English who practises alone receives no structured account of what went wrong. Conversational agents provide practice but not assessment; grammar checkers assess written text; pronunciation applications drill a fixed inventory of sounds and do not connect the drill to what the learner actually said. The result is practice without diagnosis: the learner repeats the same tense error, the same limited vocabulary and the same mispronounced phoneme because nothing in the loop makes them visible.

Building the missing loop is not simply a matter of calling a speech recogniser and a language model. Three difficulties have to be solved together. The system must capture and transport speech reliably enough for analysis; it must produce feedback on several dimensions without inventing errors, especially where model output is probabilistic; and it must do so within the latency and memory limits of affordable infrastructure. For learners who are not yet able to converse in English, a fourth difficulty appears: the system must accept their own language without pretending that English-language metrics apply to it.

3.2 Objectives

The project was guided by the following objectives:

1. To build a browser-based spoken conversation system in which a learner can hold a multi-turn conversation with an artificial-intelligence partner, with speech as both input and output.
2. To persist each conversation as a session with ordered turns, transcripts, word timings and audio references, so that a report can be generated after the call and revisited later.
3. To implement grammar, vocabulary and fluency analyses that report dimension-level findings and deterministic scores, keeping model judgement separate from measurement and arithmetic.
4. To implement phoneme-level pronunciation analysis that aligns recognised sounds against dictionary pronunciations and reports only recurring, confident patterns rather than isolated mismatches.
5. To support review of an uploaded English video of up to five minutes through the same analysis pipeline, without retaining the video itself.
6. To implement a language-bridge mode for Hindi, Kannada, Tamil and Telugu in which the learner is answered in simple English and receives validated English equivalents for native-language words actually used.
7. To engineer the system for replaceable providers, graceful degradation, bounded resource use and deployment on managed free-tier infrastructure, and to verify it with an offline automated test suite.

3.3 Functional Scope

The functional scope follows from the objectives and is summarised in Tables 1 and 2. Table 1 lists the three session modes and what each produces. Table 2 lists the spoken languages, their scripts and the recogniser that serves each in the deployed configuration.

Table 1: Session modes and their outputs

Mode	Input	Output
English Coach	Live spoken conversation in English	Conversation report: grammar, vocabulary, fluency and optional pronunciation
Language Bridge	Live spoken conversation in Hindi, Kannada, Tamil or Telugu	English reply, transcript, and an unscored list of English alternatives for native words used
Video Coach	Uploaded English video, at most five minutes	Completed session with one learner turn; grammar, vocabulary and fluency report

The bridge mode is deliberately not scored. Its report is a vocabulary aid, and applying English grammar or fluency metrics to a Hindi or Tamil transcript would produce numbers without meaning. The Video Coach mode is treated as a finished session rather than a separate analysis pipeline, because the analyses only require a learner turn containing speech.

Table 2: Supported spoken languages and speech-recognition routing

Code	Language	Script	Recogniser in deployed configuration
hi	Hindi	Devanagari	Groq Whisper large-v3
kn	Kannada	Kannada	Sarvam saaras:v4
ta	Tamil	Tamil	Sarvam saaras:v4
te	Telugu	Telugu	Sarvam saaras:v4
en	English (coached language)	Latin	Groq Whisper large-v3-turbo

3.4 Assumptions and Constraints

The design assumes a single learner per session, a browser with microphone permission and an HTTPS origin, and network access to the configured providers. It assumes that the learner speaks one of the supported languages for the whole of a bridge session; switching language mid-session is not modelled. It assumes that the uploaded video contains an audio track and audible English speech, and rejects the upload otherwise. It assumes that provider quotas are finite, and therefore bounds history, upload size, audio duration, analysis frequency and output length.

The principal constraints are memory and latency. The local pronunciation model requires roughly 1.6 GB of memory and about 1.9 GB of weights on disk, which exceeds the free deployment instance and is therefore an optional component. Turn-based interaction over HTTPS adds a full recognition, generation and synthesis round trip to every exchange. Both constraints shaped the architecture described in the next chapter and are revisited in the evaluation.

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 Design Approach

The system was designed around four principles, each of which resolved a specific risk. The first is separation of concerns along a one-directional dependency chain: the HTTP layer parses and serialises, the service layer orchestrates a turn, provider adapters speak to vendors, and repositories persist state. Nothing above the adapter layer imports a vendor SDK, so a change of provider does not reach business logic.

The second principle is that judgement and invariants are separated. A language model is well suited to deciding which sentence contains a tense error or which native word is worth translating, and poorly suited to counting occurrences, summing scores or guaranteeing that a quotation exists in the transcript. Model output is therefore treated as a proposal, and deterministic code decides what is allowed into a report and what score is displayed. This is the single most important design decision in the project, and it is visible in every analysis.

The third principle is graceful degradation. A missing optional key, an exhausted voice quota or an absent Sarvam subscription must reduce capability rather than break a session. Speech synthesis therefore has an ordered fallback chain, the bridge languages fall back to the default recogniser when Sarvam is unconfigured, and pronunciation can be switched off entirely. Each degradation is visible in the health endpoint or in the response, so a downgrade is never silent.

The fourth principle is that the system should be demonstrable and testable without network access. Offline implementations exist for every provider role, and setting the providers to their mock values runs the entire pipeline deterministically. This choice supports the automated test suite, but it also proves that the abstractions are real rather than nominal.

4.2 Overall Architecture

Figure 1 shows the layered architecture. The browser client captures audio and performs voice-activity detection locally, then communicates with the API over HTTPS using JSON and multipart uploads. The API layer exposes session, turn, analysis, video and health routes. The domain services implement conversation orchestration, video ingestion and the five analyses. Persistence is provided by a relational database and an audio store. Provider adapters isolate Groq, Sarvam and Deepgram, while the pronunciation model runs locally.

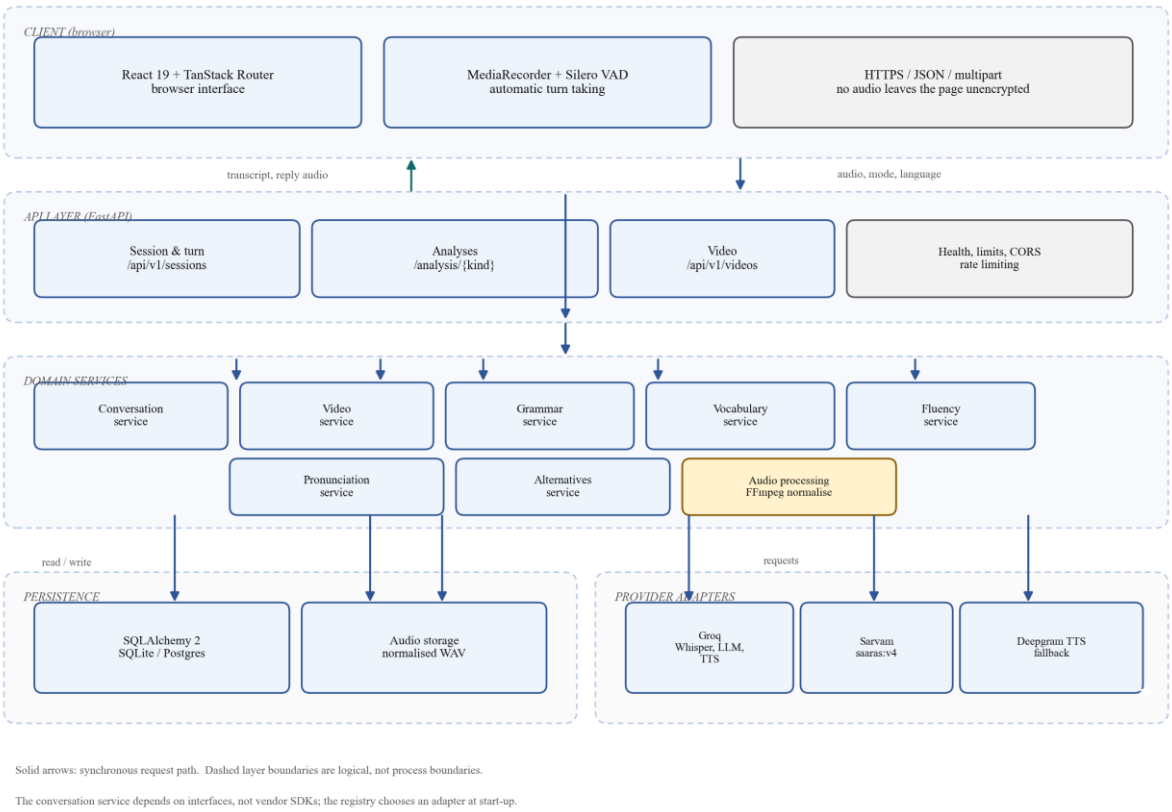


Fig. 1: Layered architecture of the system

Two properties of the diagram deserve emphasis. First, the boundaries between layers are logical rather than process boundaries: the API, services and adapters run in one Python process, and the dashed regions indicate responsibility, not deployment. Second, the conversation service depends on interfaces, not on vendor SDKs. The registry chooses a concrete adapter at start-up from configuration, and the service receives the resulting object through dependency injection.

4.3 Provider Abstraction and Registry

Seven provider roles are defined as Python protocols: speech-to-text, language model, text-to-speech, grammar analysis, vocabulary analysis, pronunciation and English alternatives. Each role has at least one hosted implementation and one offline implementation. The registry is the only place that maps a configured name, such as “groq” or “wav2vec2”, to a class. This keeps configuration knowledge in one file and makes the addition of a provider a local change.

Speech-to-text has a second dimension because the bridge requires per-language routing. A routing adapter holds a default provider and a dictionary from language code to provider. The registry constructs the router only when the default provider is not the mock, the routed-language list is non-empty and the Sarvam key is present; otherwise the default serves every language and a warning is logged at start-up. The conversation service is unaware of the routing; it passes the session's spoken language to a single interface.

Text-to-speech uses an ordered fallback chain rather than a single provider. The primary is tried first; if it fails with a provider error, the next configured provider is attempted, and the

response records which provider actually served the audio. Providers without credentials are skipped rather than treated as failures, so listing an unconfigured fallback is safe. This mechanism was added after observing that free-tier voice quotas are the most likely cause of a failed demonstration.

4.4 Lifecycle of a Conversational Turn

A turn begins when the browser posts a recorded audio segment. The backend validates the declared type and the actual byte size, normalises the audio to 16 kHz mono WAV with FFmpeg [17], stores the normalised file, and sends it to the configured speech-to-text provider with the session's language explicitly pinned [18]. The resulting transcript and word timings are stored on the turn. The service then loads a bounded window of previous exchanges, constructs the message list, and requests a reply from the language model [19]. The reply text is synthesised, the audio is stored, and the complete exchange is persisted as one turn. The HTTP response carries the transcript, the reply text, the reply audio URL, the provider that served the audio, the number of history messages sent, and a breakdown of timings.

Figure 2 expresses this as a sequence diagram. The numbering makes the critical path explicit: recognition, history construction, generation and synthesis are sequential, and persistence follows synthesis so that a turn is never stored without its reply. The figure also shows that the model sees only the bounded history window; the complete transcript remains in storage for the analyses.

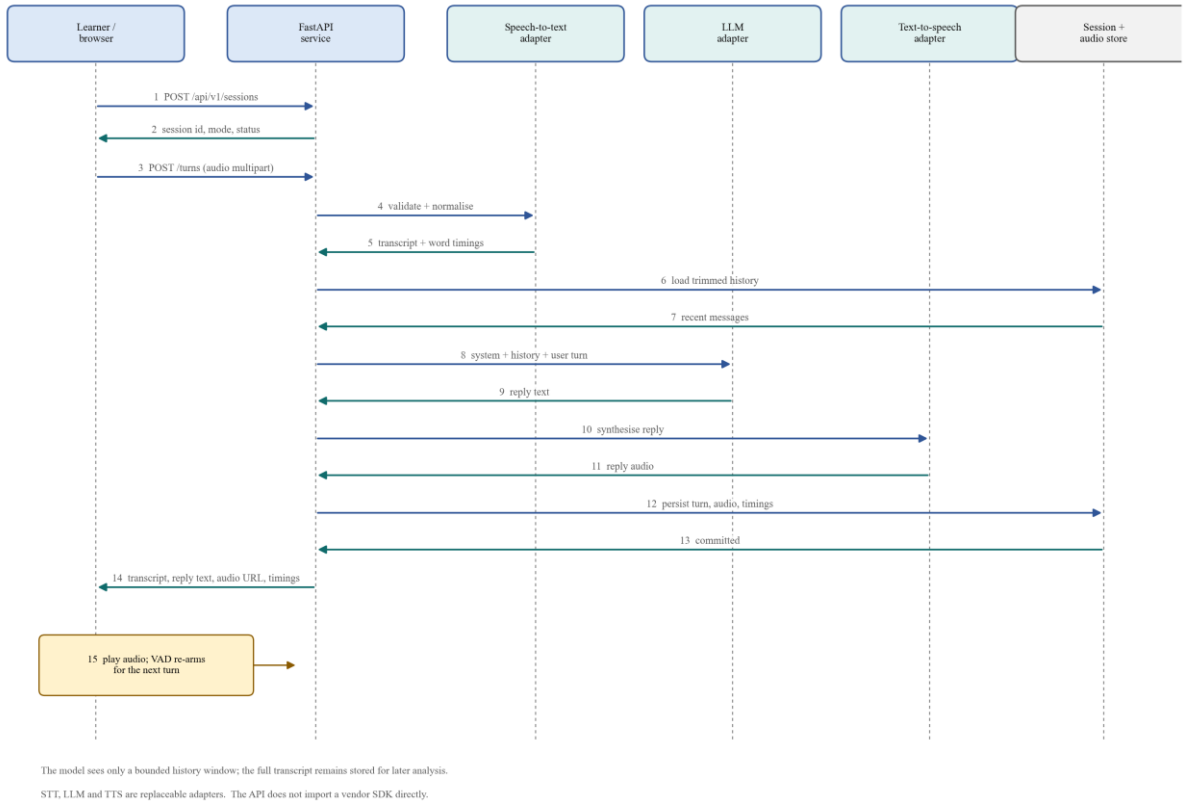


Fig. 2: Sequence of messages for one conversational turn

Two design details are worth recording. Transcription pins the language on every request instead of relying on auto-detection, because auto-detection mislabelled Kannada as Tamil and

Telugu as Hindi during development. And the normalisation step is executed in a worker thread, because a blocking FFmpeg subprocess would otherwise stall the event loop for every concurrent request.

4.5 Conversation Context Management

A five-to-eight-minute conversation can contain dozens of exchanges. Replaying all of them on every turn would grow cost and latency without bound, so two limits apply: at most twelve recent exchanges and at most eight thousand characters. When the character budget is exceeded, the oldest exchanges are dropped until the history fits. Trimming always removes a complete exchange, never half of one, so the model is never shown a reply whose question has disappeared. Each turn response reports the number of messages actually sent, which makes the context window observable rather than assumed.

The partner is instructed to reply briefly, to end with a follow-up question and not to correct the learner's mistakes during the conversation. Correction is the job of the analyses. That separation keeps the conversation natural and prevents the partner from turning every turn into a lesson, while ensuring that the report, not the dialogue, carries the diagnosis.

4.6 Session Lifecycle

A session begins in the active state. It becomes completed when the learner ends it, an operation that is idempotent, and becomes expired when no turn has been received for thirty minutes. Idleness is measured from the last turn rather than from session start, so a long conversation never times out while it is in progress. Only an active session accepts new turns; a completed session returns a conflict error and an expired session returns a gone error. Both remain fully readable, because the transcript is the input to every later analysis and ending a conversation must not make it inaccessible.

4.7 Audio Ingestion and Normalisation

Every audio path converges on the same normalisation routine. The uploaded file is written to a temporary location while its size is checked against a 25 MB limit, its duration is probed with FFprobe, and FFmpeg converts it to a single-channel 16 kHz signed 16-bit PCM WAV, discarding any video stream [17]. Audio that exceeds five minutes is rejected. Browser recordings sometimes arrive without a declared duration because MediaRecorder writes a streaming container; such files are accepted and measured after conversion rather than rejected on a missing header. The normalised file is what the speech recogniser receives, what the pronunciation model receives, and what the learner can download later.

Nine container formats are accepted for audio and six for video, which covers the formats produced by common browsers, phones and presentation software. A mislabelled file is detected during probing and reported as invalid audio rather than being passed to a provider.

4.8 Language Bridge

The bridge reuses the conversation machinery with three differences. The session is created with the bridge mode and an explicit spoken language; the recognition request is pinned to that

language; and the partner is instructed to answer in simple English only, never in the learner's language and never in mixed script. The reply is therefore immediately useful as a model of how the learner's meaning could be expressed in English, and it is also played as speech.

Kannada, Tamil and Telugu are transcribed by Sarvam's saaras:v4 model, which takes a BCP-47 language code as an explicit input and is trained with Indian languages at the centre of its distribution [20]. Hindi and English remain on Whisper, because Whisper is strong on both and there was no measured reason to move Hindi [18]. The routing is configuration-driven: adding or removing a language code changes the route without a code change, and an absent Sarvam key leaves all languages on the default provider.

At the end of a bridge session, the alternatives analysis asks the language model for English equivalents of native-language words and phrases the learner used. Every proposal then passes a deterministic validation pipeline: the English side must be non-empty and written in Latin script; the native side must occur in the transcript within a similarity of 0.8 and must not exceed six words; it must contain at least one character from the language's Unicode block and no Latin letter; it must not consist entirely of known English loanwords; and duplicates and self-translations are removed. Survivors are ranked so that common native words come first and are capped at twelve. The reported native side is always replaced by the exact transcribed span, so the learner never sees a tidied or invented spelling. The result is unscored and presented with a short recognition caveat.

4.9 Video Review

Video review accepts files up to 200 MB and five minutes. The upload is streamed to a temporary file one megabyte at a time while the received byte count is enforced, rather than trusting a declared content length. The first audio track is extracted and normalised, transcribed with English pinned, and the transcript is checked for speech. A successful ingestion produces a completed session in the video mode with one learner turn and no assistant reply; the video file is deleted, and only the normalised audio, transcript and timings are retained. Ingestion makes no language-model or synthesis call, so a slow provider does not hold the upload open longer than transcription requires.

4.10 Reliability, Limits and Error Handling

Provider failures are translated at the adapter boundary into a small set of application errors, so that no vendor-shaped payload or HTTP status reaches the service layer. The API returns a single error envelope containing a stable code, a human-readable message and optional details. Codes distinguish invalid or oversized input, missing or inactive sessions, inapplicable analyses, provider unavailability, timeouts, configuration errors and internal failures. A catch-all handler ensures that an unexpected exception never leaks internal detail to the client. Synthesis failures are contained by the ordered fallback chain described in Section 4.3; in the deployed configuration Deepgram provides the primary voice and Groq the first fallback [21].

Rate limiting is implemented as a pure ASGI middleware with a sliding sixty-second window and three buckets: twenty turns per minute, ten analyses per minute and one hundred and twenty general requests per minute. Health checks and preflight requests are exempt. The client

address is taken from the forwarded header using a configurable number of trusted proxy hops, reading from the right-hand end, because the left-hand end can be forged. A limited request receives a 429 response with a retry hint and with CORS headers intact, which required adding the limiter before the CORS middleware so that it is the inner layer.

Cross-origin access is restricted to configured front-end origins, with an optional regular expression for preview deployments. Credentials are not allowed, because the application uses no cookies and no authentication headers.

4.11 Data Storage Approach

A session is stored as one row, its turns as child rows, and its analysis as a single row with one nullable section per analysis. Findings are stored as child rows so that they can be queried later, for example to ask which filler words recur. Audio is not stored in the database: turn rows hold the storage key, format, duration and size, while the bytes live behind a storage interface that writes to local disk in the current deployment. This keeps the database small and allows the audio store to be replaced by object storage without a schema change.

The repository interface has two implementations, an in-memory store and a SQL store, and both are verified against the same contract test suite. The SQL implementation uses SQLAlchemy's asynchronous engine and, in production, a managed PostgreSQL instance. Because SQLite does not preserve time zones, a custom column type normalises timestamps to UTC on write and restores them on read; without it, the idle-expiry check would compare naive and aware datetimes.

5. PROPOSED ALGORITHMS

5.1 Principle: Model Proposes, Code Decides

All five analyses follow the same pattern. The language model or acoustic model produces candidate findings or measurements; deterministic code then validates, filters, aggregates and scores them. The pattern is not stylistic. Language models vary between runs, miscount, and occasionally quote text that was never said; acoustic models produce spurious substitutions at accents and word boundaries. Placing scoring and evidence checks in code makes the output reproducible for a given input and makes every number explainable to the learner.

5.2 Grammar Analysis

Only learner turns are analysed; the partner's replies are excluded. The learner's text is split into sentences at sentence-final punctuation, and each sentence is sent to the configured grammar provider, which returns issues with a category, a quotation, a correction, an explanation and a confidence. Eight categories are used: tense, articles, prepositions, subject-verb agreement, plurals, word form, sentence structure and other. Because the input is transcribed speech, the provider is instructed not to report punctuation or capitalisation.

The returned issues are filtered before display. Findings below a confidence of 0.4 are discarded, duplicates are removed, and no more than twenty-five are reported. A reduction step then removes a finding whose quotation contains an already-reported narrower quotation of the same category, and removes a broad finding that merely restates two or more findings already made. This handles the observed tendency of a model to report both “I go to college yesterday” and the narrower “go” for the same tense error.

Scoring is performed in code and groups issues by sentence, so that a sentence containing three mistakes counts once as a flawed sentence rather than three times. Each flawed sentence costs the confidence of its strongest finding, and the score is

$$score = 100 - min(1, \Sigma\, w(s) / S) \times 100$$

where S is the number of analysed sentences and w(s) is the worst confidence among the findings attached to sentence s. A conversation with no findings scores 100, because absence of evidence of error is not evidence of poor grammar. The transcript itself is never rewritten; every issue carries its own copy of the quoted sentence, and a test asserts that the stored transcript is byte-identical before and after analysis.

5.3 Vocabulary Analysis

Counting is performed in code, and the provider is asked only for suggestions. This division follows from the observation that “you said this phrase four times” is arithmetic, and that a model asked to count will sometimes be wrong. The analysis detects four issue types: repeated words, repeated phrases, over-used basic words, and expressions that are grammatical but unnatural.

Tokens are lower-cased and stripped of surrounding punctuation. Stop words are excluded from repetition counts. The repetition threshold scales with conversation length as max(3, round(W / 60)), where W is the number of analysed words, so that three occurrences in eighty words count as a habit while three in eight hundred do not. Phrases of two and three words are counted after trimming stop words from both ends, so the learner is told about “very good” rather than “was very good”; a three-word phrase that recurs at least twice suppresses the two-word phrases it contains, so the same habit is not reported twice. A list of twenty-one basic words, including “good”, “bad”, “nice”, “thing”, “stuff”, “very” and “really”, is flagged at three occurrences. Suggestions are capped at four per issue and issues at twelve.

The vocabulary score charges only repetition beyond the threshold, because using a word up to its threshold is normal speech. When the excess reaches fifteen per cent of all words spoken, the score reaches zero. Lexical diversity, measured as the type-token ratio, is reported but deliberately not scored: the ratio falls as any text grows longer, so scoring it would penalise a learner for speaking more.

5.4 Fluency Analysis

Fluency is measured rather than judged. It consumes the word timings that the recogniser returns with the transcript, which are stored on the turn at transcription time; re-transcribing later would spend another provider call on data already held. All timings are accumulated per learner turn, never across turns, so that silence while the partner is speaking is not attributed to the learner.

A gap of at least 0.5 seconds between consecutive words is counted as a pause and a gap of at least 1.0 second as a long pause. Speaking rate is the number of words divided by the analysed seconds, expressed in words per minute, and articulation rate excludes pause time. The comfortable band is 110 to 160 words per minute. Filled pauses are counted from a list of single tokens, including “um”, “uh”, “erm”, “er”, “ah”, “hmm”, “mm” and “eh”, and from multi-word phrases such as “you know”, “I mean”, “sort of”, “kind of” and “what do you call it”; a small set of soft markers, including “like”, “actually”, “basically” and “literally”, is counted separately. An immediate repetition of the same word is counted once, and a restart is identified when the same two-word pair recurs within six tokens in the same turn, with overlapping hits counted once. At most fifteen findings are reported.

The score has four equally weighted parts, each capped at twenty-five points: rate outside the comfortable band, long pauses per minute, fillers per hundred words and disfluencies per hundred words. No single habit can therefore sink the score alone, and the arithmetic remains explainable. A typed conversation returns a null fluency score with an explanation rather than a fabricated number.

5.5 Pronunciation Analysis

Pronunciation analysis compares the sounds a speaker produced with the sounds their words require. Two sequences are built independently, as shown in Figure 3. On the reference path, the transcript is converted to phonemes by g2p_en, which uses the CMU Pronouncing Dictionary where possible and a neural model otherwise; ARPAbet symbols are then mapped

to IPA through a thirty-nine-entry table. On the acoustic path, the normalised 16 kHz audio is passed to a wav2vec2 model fine-tuned for phoneme recognition, which emits IPA symbols directly. The tokenizer is loaded with phonemisation disabled, so no espeak installation is required.

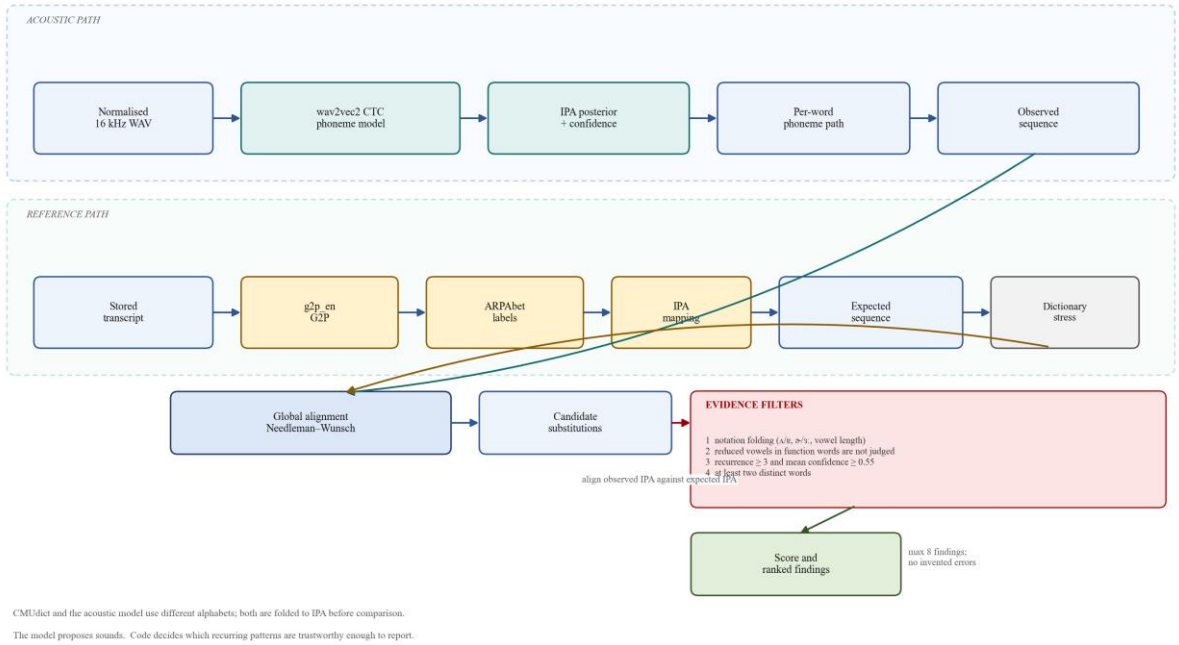


Fig. 3: Pronunciation analysis pipeline: acoustic and reference paths, alignment and evidence filters

The acoustic model classifies every twenty milliseconds and produces a posterior distribution over phonemes. The decoder takes the most probable label per frame, collapses runs of identical labels and removes blanks, in the standard CTC manner, and records the mean posterior probability of each surviving phoneme as its confidence. The two sequences are then aligned globally with the Needleman-Wunsch algorithm using unit substitution and gap costs, which yields the minimum-cost edit script between them. Alignment, rather than position-by-position comparison, prevents a single insertion or deletion from shifting every subsequent phoneme and producing a cascade of false errors.

Mismatches from the alignment are candidate substitutions. Before aggregation, two corrections are applied. Notation folding treats transcription differences as equivalent, for example /ʌ/ and /ɐ/, /ɜː/ and /ɚ/, and flapped or glottal variants of /t/ and /ɾ/; these are properties of the dictionaries and the acoustic model, not mispronunciations. Reduced vowels in a list of forty-one function words are not judged, because nobody pronounces “and” with the vowel a dictionary gives it; consonants in those words are still assessed, otherwise almost all evidence for /ð/ would disappear.

The surviving substitutions are aggregated per phoneme pair across the whole conversation. Four evidence filters then decide what may be reported: a pattern must occur at least three times, its mean confidence must be at least 0.55, it must appear in at least two different words, and at most eight findings may be shown, each with at most six example words. Findings are ordered by occurrence count and then by confidence.

The score is

$$score = 100 - \min(1, (\Sigma\, occurrences \times confidence / W) / 0.25) \times 100$$

where W is the number of analysed words and the inner sum runs over reported patterns. A learner whose speech produces no trustworthy pattern therefore scores 100, and a pattern that affects a quarter of all words drives the score to zero. Practice suggestions are drawn from a small hand-written list of drill words for twenty-seven phonemes, for example “think, three, thank, path, birthday” for /θ/. The wording is deliberately advisory, describing a sound as worth practising rather than declaring it wrong.

5.6 English Alternatives Validation

The alternatives analysis is the clearest example of the propose-and-verify pattern. The model is given the learner's utterances, the language name and script, speech notes and a sample of known loanwords, and is asked for a JSON list of native-language expressions with English equivalents. Each proposal is then subjected to six checks in order: the English side must be non-empty and written in Latin script; the native side must match a real transcribed span within a similarity of 0.8 using a sliding window of the quoted length plus or minus one word; it must contain no Latin letter and at least one letter from the language's Unicode block; it must not consist entirely of loanwords, ignoring function words so that a loanword carrying a postposition is still recognised; it must not duplicate an earlier item or translate to itself; and it must not exceed six words. Survivors are ranked by whether they contain a common native word, and the list is capped at twelve. The matched span replaces the model's quotation, so the report shows what was heard.

5.7 Thresholds and Constants

Table 3 collects the thresholds that determine behaviour. They are defined as named constants in the source and covered by tests, so that a change is deliberate rather than accidental.

Table 3: Principal analysis thresholds and limits

Analysis	Constant	Value and effect
Grammar	Minimum confidence	0.4; findings below this are discarded
Grammar	Maximum issues	25 per conversation
Vocabulary	Repetition threshold	max(3, round(words / 60)) occurrences
Vocabulary	Basic-word threshold	3 occurrences before flagging
Vocabulary	Excess for zero score	15 per cent of all words
Fluency	Pause / long pause	0.5 s / 1.0 s
Fluency	Comfortable rate	110–160 words per minute
Fluency	Maximum findings	15
Pronunciation	Minimum occurrences	3 per phoneme pattern
Pronunciation	Minimum mean confidence	0.55

Analysis	Constant	Value and effect
Pronunciation	Minimum distinct words	2
Pronunciation	Maximum issues	8
Alternatives	Transcript match similarity	0.8 (difflib ratio)
Alternatives	Maximum span length	6 words; at most 12 items reported

5.8 Computational Cost

The deterministic parts are inexpensive relative to the provider calls. Grammar and vocabulary processing are linear in the number of words apart from phrase counting, which is linear in the number of phrase positions with a small constant for lengths two and three. Fluency is a single pass over word timings. Pronunciation is dominated by the acoustic model, not by the algorithms: alignment is O(mn) in the lengths of the expected and observed phoneme sequences, which for a sentence of twenty words is a few hundred symbols and therefore negligible. The model itself runs at roughly 1.35 times real time on four CPU threads in the reported configuration, and its weights are loaded once and held for the life of the process.

6. PROPOSED FLOWCHART, BLOCK DIAGRAM AND DATA-FLOW DIAGRAMS

This chapter presents the data-flow and workflow views of the system. The layered block diagram appears as Figure 1 in Chapter 4, where it supports the architectural discussion; the diagrams here show the same system from the perspectives of data movement and user workflow.

6.1 Context Diagram (Level 0)

Figure 4 is the level-0 data-flow diagram. The learner is the only human actor. The system exchanges audio, control information and reports with the learner's browser; requests and responses with external speech and language providers; and records with the database and the audio store. An uploaded video is shown as a separate external input because it enters the system without a microphone and without a live conversation.

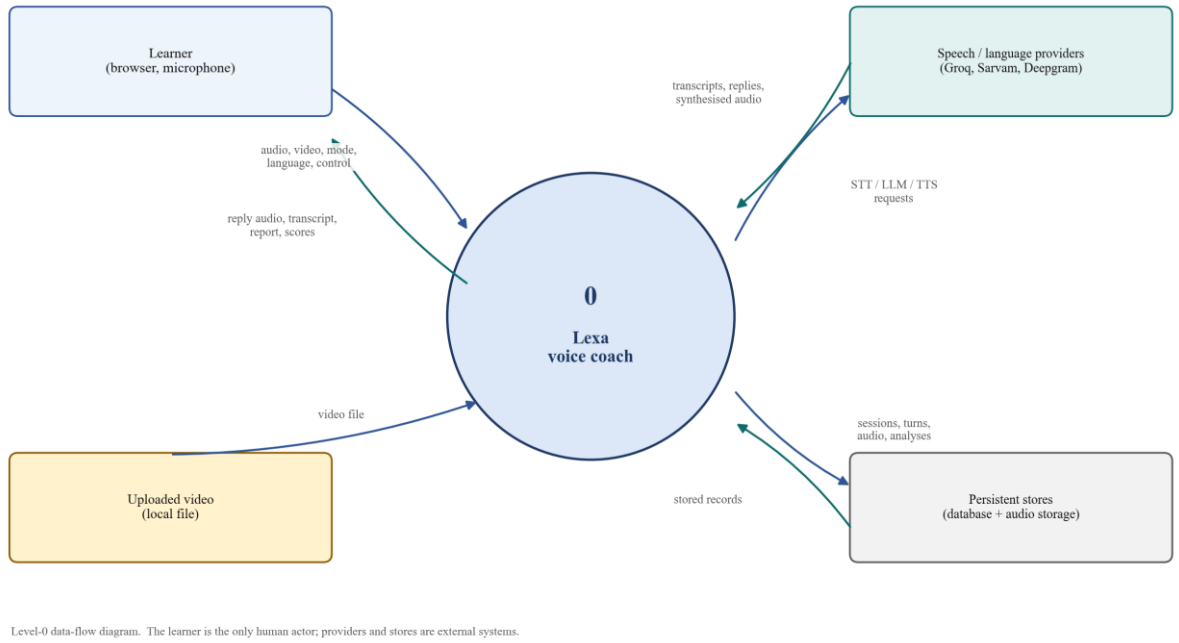


Fig. 4: Level-0 data-flow diagram (context diagram)

The diagram makes an important boundary explicit: the providers are external systems, not parts of the application. The application owns the orchestration, the validation and the stored record; the providers own recognition, generation and synthesis. That separation is what allows a provider to be replaced without changing the flows that the learner experiences.

6.2 Level-1 Data-Flow Diagram

Figure 5 decomposes the system into six processes and three data stores. Process 1 captures and validates incoming audio. Process 2 transcribes it and generates the reply. Process 3 persists the turn and its audio. Process 4 analyses English speech. Process 5 builds the language bridge. Process 6 ingests a video. The stores hold the session and turn records (D1), the analysis records (D2) and the audio files (D3).

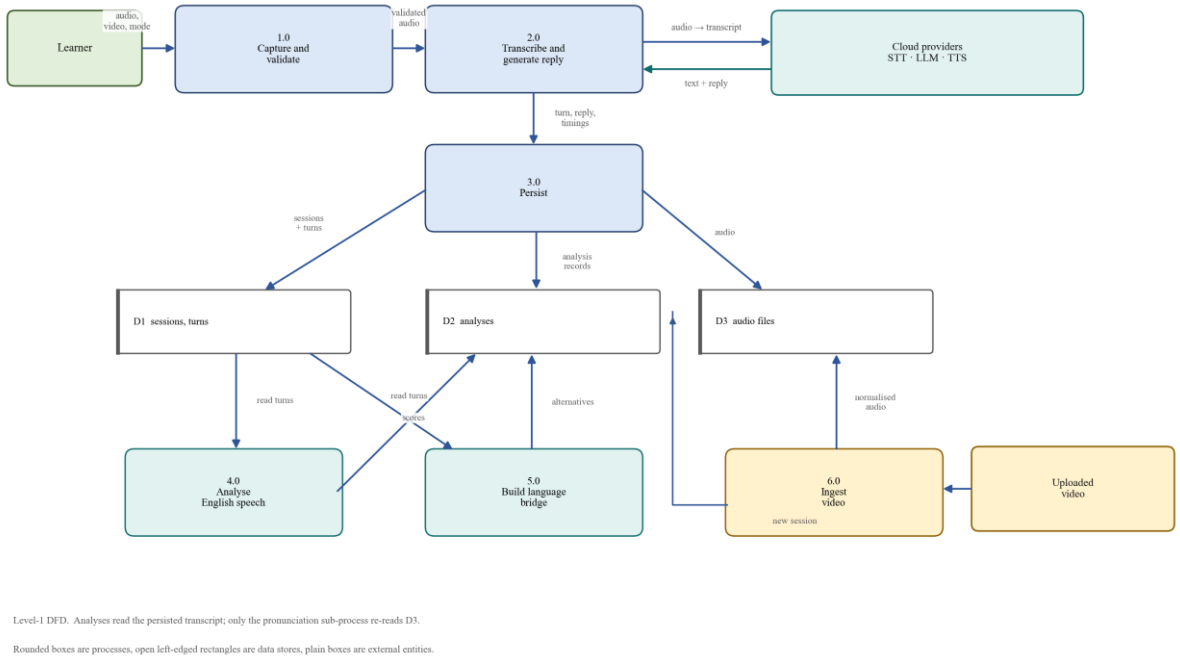


Fig. 5: Level-1 data-flow diagram

Two flows in the diagram are easy to miss but matter for correctness. The analysis processes read stored turns rather than the original upload, which means that a report can be regenerated without the learner repeating the conversation; only pronunciation re-reads the stored audio, because it is the only analysis that needs the signal itself. And video ingestion writes a completed session, after which the ordinary analysis processes run unchanged. The dashed annotations clarify that rounded boxes are processes and open-ended rectangles are data stores, following the usual convention.

6.3 Video Review Workflow

Figure 6 is a flowchart of video review. The file's extension and size are checked before it is streamed to a temporary file; the duration is checked after probing; FFmpeg extracts a 16 kHz mono track; recognition is attempted with English pinned; and an empty transcript is rejected as containing no speech. A successful path ends in a completed session with one learner turn, from which the client can run grammar, vocabulary and fluency analyses. The rejection paths carry distinct status codes so that the interface can explain what was wrong rather than showing a generic failure.

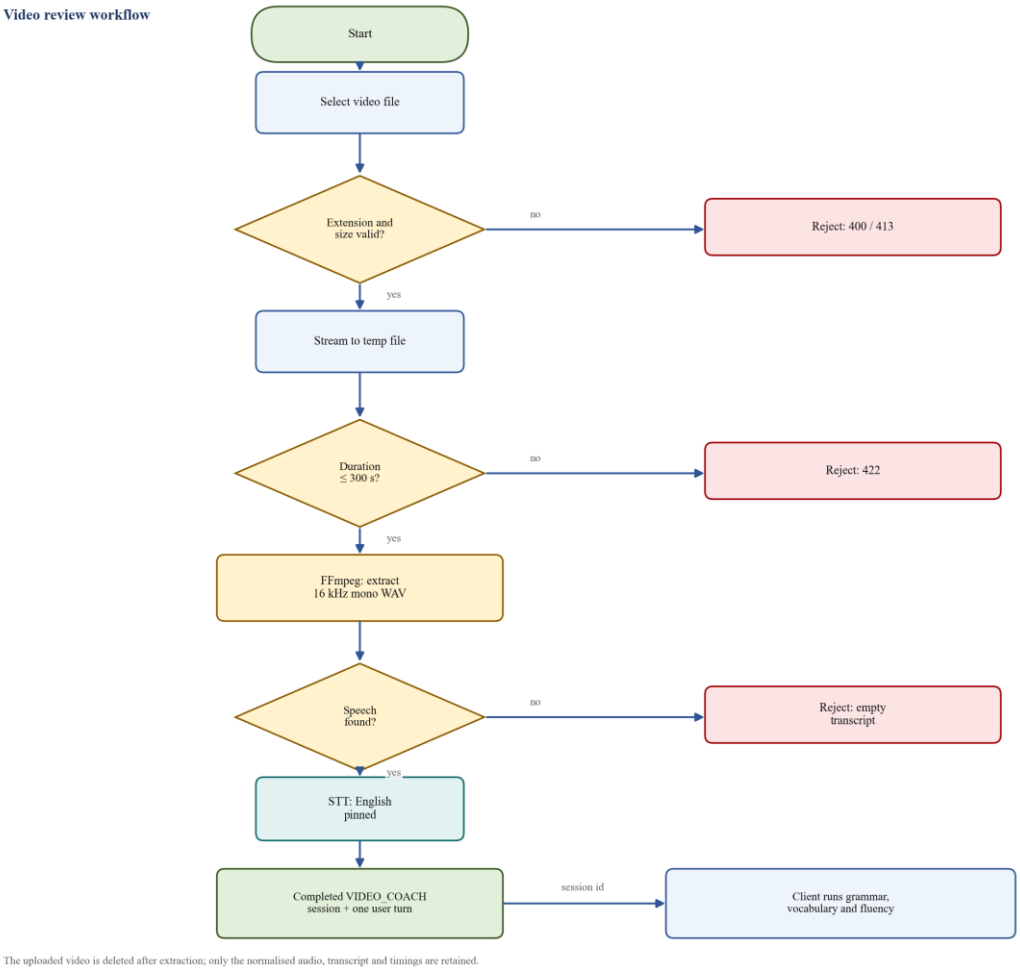


Fig. 6: Video review workflow

The flowchart also records a privacy-relevant decision: the uploaded video is deleted after extraction, and only the normalised audio, the transcript and the word timings are retained. The system therefore stores what the analyses need, which is not the video.

6.4 Language-Bridge Workflow

Figure 7 shows the bridge workflow from the learner's choice of language to the final alternatives report. The language determines the recogniser: Kannada, Tamil and Telugu are routed to Sarvam, while Hindi and English use Whisper. The recognised text is sent to the language model with a bridge-specific prompt that requires an English-only reply, and the reply is synthesised and played. The native-language alternatives are produced separately at the end of the session and pass through the validation and ranking stages described in Chapter 5.

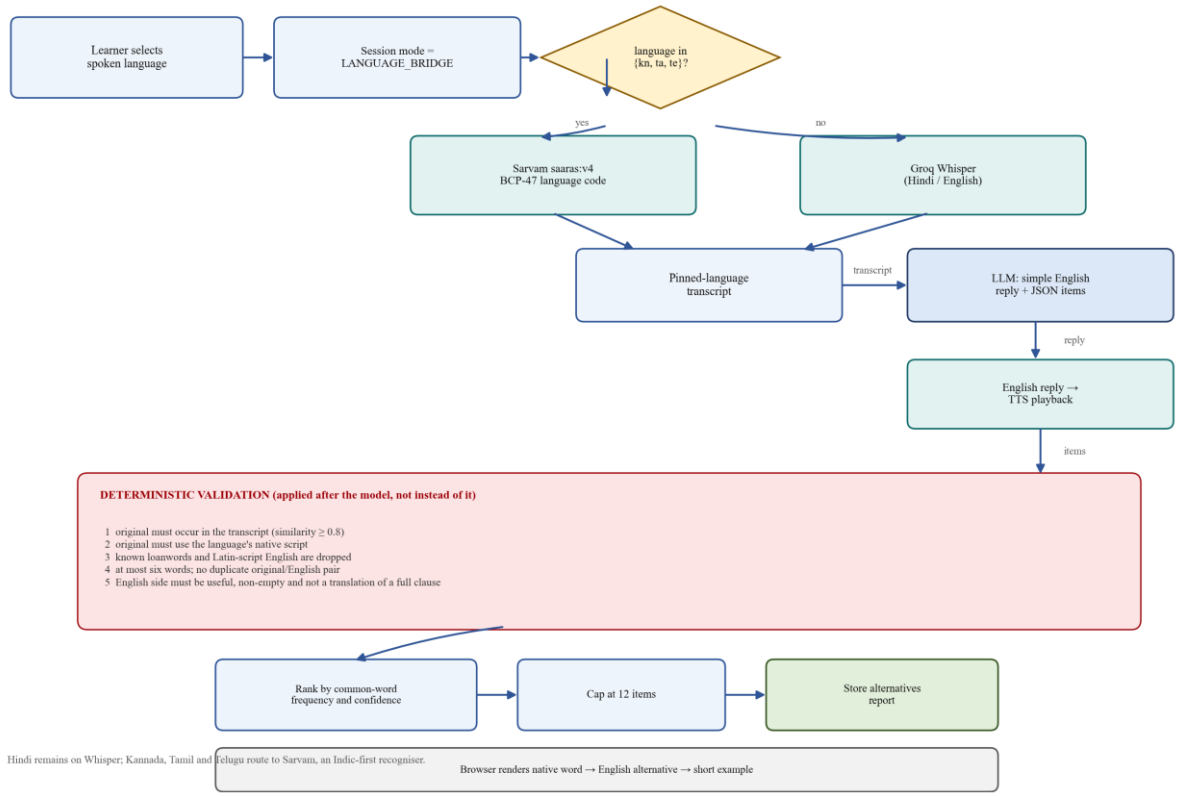


Fig. 7: Language-bridge workflow, including deterministic validation of alternatives

The figure groups the validation checks into a single stage because they are applied as a pipeline to every candidate. This is deliberate: the model's output is one input to the stage, not the stage's authority. The checks are what guarantee that the left-hand side of every reported pair is really the learner's language and really appeared in the transcript.

Taken together, the diagrams describe a system in which the learner's speech is normalised once, stored once, and then analysed from several independent perspectives. The next chapter documents how that design is realised in code, in the database, in the browser and in the deployment.

7. IMPLEMENTATION: TOOLS, TECHNOLOGY AND CODE

7.1 Technology Stack

The implementation is deliberately conservative in its choice of frameworks. The backend uses FastAPI because its dependency-injection system maps naturally onto the provider protocols described in Chapter 4, and because it generates an OpenAPI schema that documents the entire API without extra work [22]. Data access uses SQLAlchemy 2.0's asynchronous interface, which supports both SQLite for local development and PostgreSQL in deployment without changing the repository code [23]. The front end uses React 19 with TanStack Start for server-rendered routing, Vite for the build, Tailwind CSS for styling and a set of accessible Radix-based components [24], [25]. Table 4 lists the principal dependencies and their roles.

Table 4: Principal technologies and their roles

Layer	Technology	Role in the system
Language	Python 3.12	Backend implementation and analysis code
Web framework	FastAPI 0.141, Uvicorn 0.52	HTTP API, dependency injection, OpenAPI schema
Validation	Pydantic 2.13, pydantic-settings	Request models and environment configuration
Persistence	SQLAlchemy 2.0 (async), aiosqlite, asyncpg	Session, turn and analysis storage on SQLite or PostgreSQL
HTTP client	httpx 0.28	Provider clients with explicit timeouts and error translation
Media	FFmpeg and FFprobe	Duration probing and conversion to 16 kHz mono WAV
Speech models	PyTorch 2.14 (CPU), Transformers 5.16, g2p_en 2.1	Optional local phoneme recognition and expected-phoneme generation
Front end	React 19.2, TanStack Start and Router, Vite 8.2	Server-rendered interface, routing and build
Styling	Tailwind CSS 4.2, Radix-based components	Layout, typography and accessible controls
Browser audio	MediaRecorder, @ricky0123/vad-web 0.0.31, onnxruntime-web 1.30	Recording and in-browser voice-activity detection
Testing	pytest 9.1, pytest-asyncio, respx 0.23	Offline unit and integration tests with mocked HTTP
Deployment	Docker, Render, Neon PostgreSQL, Vercel	Container build, API hosting, managed database, front-end hosting

Three choices deserve justification. The speech models are an optional dependency group rather than a base dependency, so the container that serves conversation requests never installs PyTorch or Transformers; this keeps the image small enough to start quickly and avoids shipping half a gigabyte of wheels that most requests never use. PyTorch is pinned to its CPU-only index, because a CUDA build would add several gigabytes for a workload that runs on four threads. And the audio pipeline depends on FFmpeg rather than a Python decoder, because FFmpeg handles every container and codec that a browser, phone or camera can produce, and its probing output distinguishes a genuinely invalid file from one merely mislabelled [26].

7.2 Backend Structure

The backend is organised by responsibility. The application factory wires middleware, exception handlers and startup checks. Configuration is a single settings object populated from the environment. Domain models are plain Pydantic objects, independent of the database. Routes are thin: they parse input, call a service and serialise the result. The conversation service owns the turn orchestration; the video service owns ingestion; the analysis modules own measurement and scoring; the provider package owns everything vendor-specific; and the session, storage and database packages own persistence.

Startup performs three useful checks. It warns if FFmpeg is absent, if a hosted provider is selected without its key, or if bridge languages are routed to Sarvam without a Sarvam key. It constructs the provider registry once and injects the resulting object graph into the routes. On shutdown it closes HTTP clients and disposes of the database engine. These checks turn a class of configuration mistakes from a failed request into a clear log line at boot.

7.3 API Surface

Table 5 summarises the twenty-two routes. All application routes are versioned under /api/v1 except health probes, which are intentionally unversioned so that infrastructure can monitor them. Errors share one envelope with a stable code, and the OpenAPI page at /docs provides an interactive reference.

Table 5: API surface

Area	Routes	Purpose
Health	GET /health; GET /health/providers	Liveness, FFmpeg presence, configured providers and active language routes
Sessions	POST /api/v1/sessions; GET /api/v1/sessions/{id}	Create a session in a chosen mode; read its state and turns
Turns	POST .../turns; POST .../turns/text	Upload spoken audio or a typed turn; receive transcript, reply and audio URL
Transcript	GET .../transcript	Complete transcript, optionally as plain text
Audio	GET .../turns/{turn_id}/audio; GET .../audio/user	Assistant reply audio; the learner's own normalised audio

Area	Routes	Purpose
Session end	POST .../end	Complete a session; idempotent
Analyses	GET and POST .../analysis/grammar, vocabulary, fluency, pronunciation	Run or retrieve each stored analysis
Bridge	GET and POST .../analysis/english-alternatives	Run or retrieve the unscored alternatives report
Video	GET /api/v1/videos/limits; POST /api/v1/videos/transcribe	Report limits; ingest a video as a completed session

7.4 Database Design

Figure 8 shows the entity-relationship model. Nine tables are defined. A session has many turns, and each turn stores its speaker, transcript, audio reference and word timings. A session has exactly one analysis row, which contains nullable sections for grammar, vocabulary, fluency and pronunciation; this design lets each analysis be written independently and in any order. Findings are child rows of the analysis: grammar issues, vocabulary issues, fluency findings and pronunciation issues. The bridge adds two tables, one row per alternatives report and one per item.

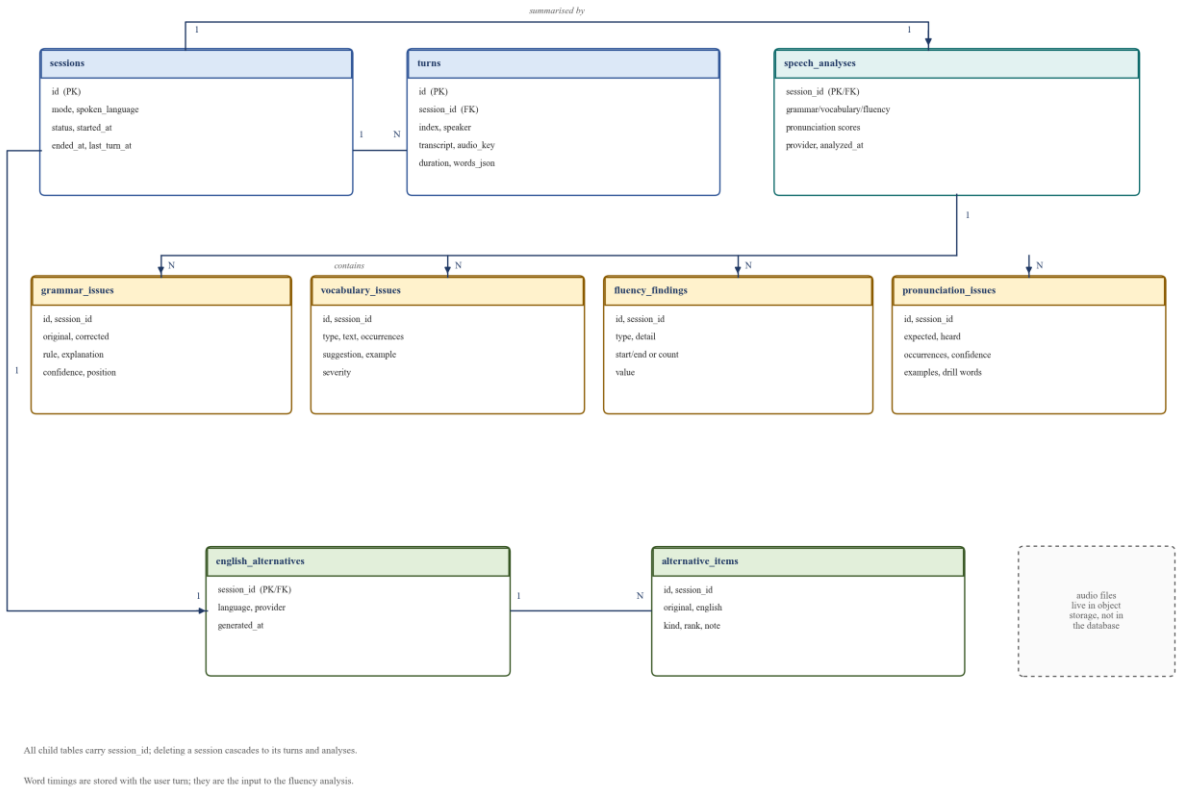


Fig. 8: Entity-relationship model of the persistence layer

Word timings are stored as JSON on the learner's turn because they are a variable-length list consumed as a whole by the fluency analyser, not queried field by field. Audio is referenced rather than embedded: the turn holds a storage key, format, duration and size, while the bytes live in the audio store. Deleting a session cascades to its turns and analyses, and the audio keys

are removed through the storage interface. Foreign keys are indexed, and the pronunciation issues table is indexed on the expected phoneme so that recurring sound patterns can be queried across sessions in future work.

The schema is created at start-up if absent, and a lightweight compatibility step adds any missing nullable columns. That is sufficient for a project whose schema evolved during development, but it is not a substitute for migrations; a production system would use a migration tool, and this is noted as a limitation.

7.5 Front-End Implementation

The front end is a server-rendered React application with five file routes: a landing page, a live-conversation page, a video page, an insights page and an alternatives page. Figure 9 shows the route flow. The landing page explains the three-step loop, previews a sample report and offers the choice of language before any request is made. The conversation page creates a session and drives the capture-playback loop. The insights page renders the report and runs the analyses. The alternatives page renders the bridge vocabulary report.

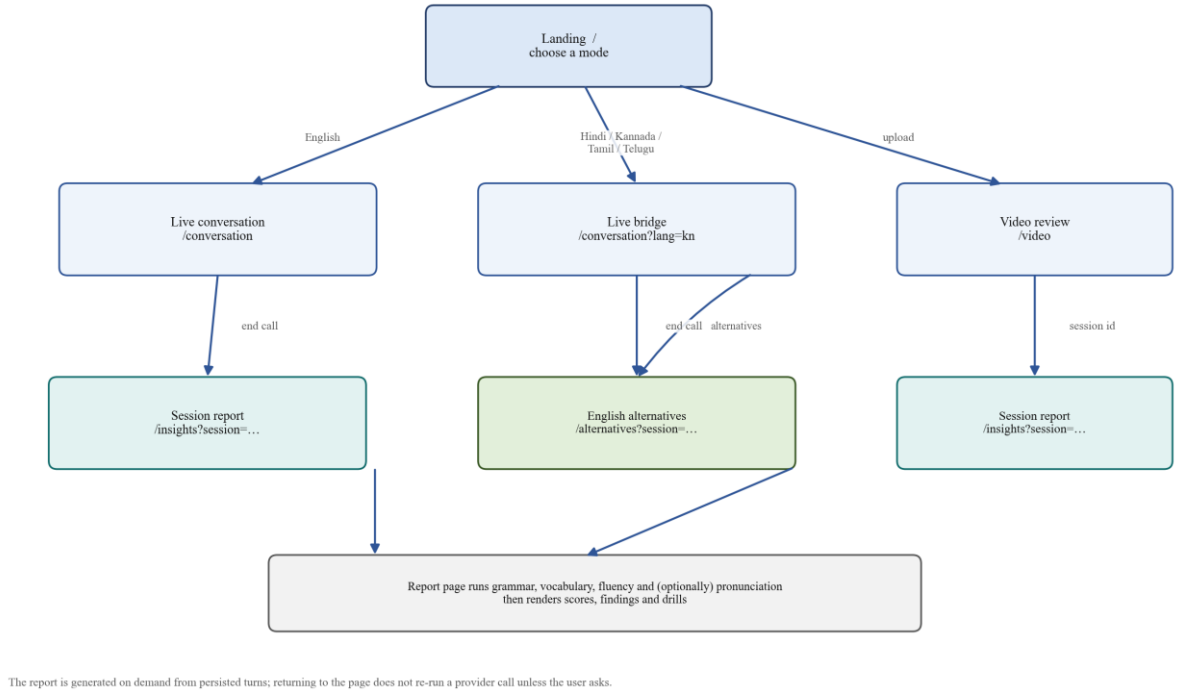


Fig. 9: Front-end route flow and report generation

Recording uses MediaRecorder with an ordered list of container preferences, beginning with WebM/Opus and falling back to MP4 for browsers that require it; the uploaded filename follows the chosen container [15]. Voice-activity detection runs Silero through a WebAssembly build of ONNX Runtime, with the model and runtime loaded from a content-delivery network because they are too large to bundle [16]. The detector waits 1,400 milliseconds of silence before ending a turn, which was chosen to avoid cutting off a speaker who pauses to think, and requires 400 milliseconds of speech to start one, so that a cough or a mouse click does not create a turn. The audio buffer produced by the detector is not uploaded: it is 16 kHz mono PCM, whereas the backend expects the compressed container, so the

detector's only job is to stop the same MediaRecorder that is already running. If the detector fails to load, the manual microphone button remains available.

The interface has explicit states for starting, idle, recording, thinking, speaking and failure. It remains in the speaking state until the reply audio's ended event fires, so that the learner cannot start a new turn over the partner's voice. The report page runs fluency, grammar and vocabulary sequentially rather than in parallel, because the backend rate-limits analyses per minute and because progressive rendering gives the learner something to read while later sections are still being produced. Pronunciation is behind a separate control, shown only when it is enabled at build time, with an expectation-setting message about the first-run model load.

All backend communication is confined to one API client module, whose TypeScript types mirror the Pydantic models. Errors are decoded from the backend envelope into a typed error carrying code, status and details, so that the interface can distinguish a rate limit from an invalid file. Video upload uses XMLHttpRequest rather than fetch because only the former reports upload progress; a video can be hundreds of megabytes, and a progress bar that does not move is indistinguishable from a hung application. Video limits are fetched from the server rather than hard-coded, so the page cannot advertise a stale duration or size.

7.6 Testing

The backend has thirty test modules. The suite is fully offline: provider dependencies are replaced through FastAPI's dependency overrides, the SQLite store and the in-memory store are tested against the same contract, generated WAV files stand in for recordings, and the hosted adapters are tested against a mocked HTTP transport. This makes the suite deterministic and free to run, and it means that a change in request shape, such as a renamed multipart field or a wrong language code, fails a test rather than a live demonstration.

Coverage is organised by behaviour rather than by file. Session lifecycle tests assert that ending is idempotent, that a completed session refuses turns but remains readable, and that idleness expires a session while activity keeps it alive. History tests assert that trimming never separates a question from its reply and that no context leaks between sessions. Vocabulary tests include a deliberately miscounting model to prove that counts shown to the learner come from code. Fluency tests separate articulation rate from speaking rate and prove that a false start is counted once. Pronunciation tests exercise alignment and the evidence filters. Language-bridge tests assert that Latin-script originals, mixed-script spans and loanwords in native script are dropped for all four languages. Rate-limit tests cover the sliding window, forged forwarding headers and the presence of CORS headers on a 429 response. A full run of the suite on the development machine completed with 454 passing tests in about one minute.

7.7 Deployment

The deployed topology is shown in Figure 10. The front end is hosted on Vercel and built with the Vercel preset of the Nitro server [29]. The API runs as a Docker container on Render, with a health check on /health and a build filter that rebuilds only when backend files change [27]. The database is a managed PostgreSQL instance on Neon, reached over TLS; the connection string is rewritten at start-up into the form the asynchronous driver expects [28]. Speech and

language providers are reached over HTTPS from the API container, and audio is written to the instance's disk.

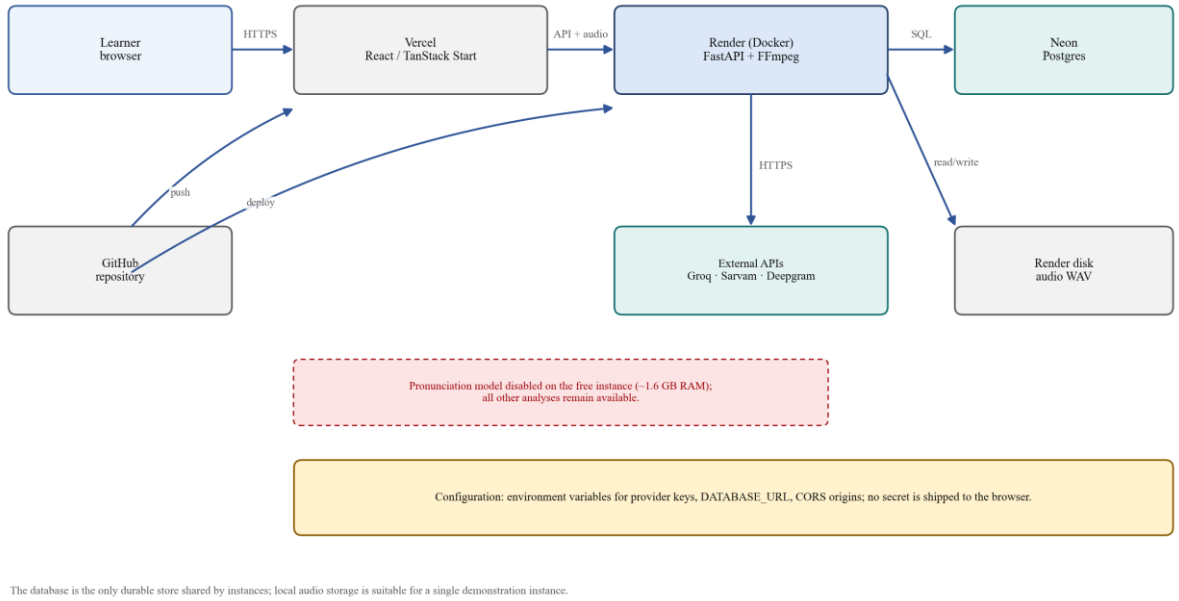


Fig. 10: Deployment topology

The container is built from a slim Python image with FFmpeg installed explicitly, the uv package manager copied from its official image, and dependencies installed from the lock file without development packages. Omitting the development group also omits the pronunciation

group, so PyTorch and Transformers never enter the image. The server binds to the port injected by the platform at runtime.

Table 6: Selected deployment configuration

Variable	Deployed value	Reason
STT_PROVIDER, LLM_PROVIDER	groq	Hosted recognition and generation with a generous free tier
TTS_PROVIDER	deepgram	Voice quota on the Groq free tier is small; Deepgram credit is larger
TTS_FALLBACK_PROVIDERS	groq,mock	Degrade to another voice, then to offline audio, rather than fail
SARVAM_STT_LANGUAGES	kn,ta,te	Route the three Dravidian languages to the Indic-first recogniser
PRONUNCIATION_PROVIDER	off	The phoneme model needs about 1.6 GB of memory, more than the free instance offers
SESSION_STORE	database	Sessions and reports must survive a restart
TRUSTED_PROXY_HOPS	1	Read the client address from the platform proxy rather than a forgeable header

The deployment is honest about its constraints. The first request after a quiet period is slow while the free instance wakes. Audio files live on the instance disk and are lost on restart, although sessions, transcripts and reports survive in PostgreSQL. Rate-limit counters are held in process memory, so they reset on restart and are not shared across instances. Pronunciation analysis is disabled, and enabling it requires a host with at least two gigabytes of memory and a front-end build with the corresponding flag. These are documented in the deployment guide rather than discovered during a demonstration.

8. RESULT DISCUSSION AND ANALYSIS

This chapter reports what was observed. The system was exercised through its API and through the browser interface, with generated and recorded audio, against the hosted providers. The numbers below are measurements from those runs, not benchmarks: they come from a small number of observations on a development machine in India and on the free deployment, and they are presented with that qualification.

8.1 Automated Verification

The offline suite is the strongest evidence of functional correctness. Thirty modules cover the API, the services, the providers, the analyses, persistence and deployment-related behaviour, and a full run of the suite reported 454 passing tests in about one minute. Table 7 groups the suites by the behaviour they protect rather than listing every module.

Table 7: Automated test coverage by behaviour

Area	Representative behaviours asserted
Conversation flow and context	Multi-turn replies, history replay, bounded history, no cross-session leakage, exchange-aligned trimming
Session lifecycle and persistence	Idempotent end, refusal after completion, idle expiry, durability across restart, audio download
Audio processing	16 kHz downsampling, WAV header repair, streamed recordings without duration, non-blocking conversion
Grammar and vocabulary	Score arithmetic, filtering, narrowest-quote preference, miscount-proof counts, threshold scaling, caps
Fluency	Rate versus articulation, pause classification, single-counted false starts, four-part score caps, null for typed text
Pronunciation	Phoneme alignment, notation folding, reduced-vowel handling, recurrence and confidence filters
Language bridge	Script and loanword rejection in four languages, transcript matching, ranking, cap, API boundaries
Providers and fallbacks	Language pinning, request shapes, error translation, routing by language, TTS fallback order
Platform behaviour	Rate-limit windows, forged forwarding headers, CORS including 429 responses, video limits, error envelopes

The suite found a class of defect that manual testing had missed. Rate-limited responses initially lacked CORS headers, so the browser reported a network error instead of the real cause; the test asserted the header, and the fix was to order the middleware so that the limiter runs inside CORS. A similar case concerned MediaRecorder files with no declared duration, which FFprobe reports as unknown; the test required them to be accepted and measured after conversion.

8.2 End-to-End Turn Latency

Table 8 reports two representative turns recorded by the service itself. The English turn used Whisper large-v3-turbo, the GPT-OSS dialogue model and Orpheus synthesis. The bridge turn used Sarvam for recognition, the same dialogue model and Deepgram synthesis.

Table 8: Observed latency for one turn (milliseconds)

Turn	Speech-to-text	Language model	Text-to-speech	Total
English conversation	518	748	702	1,968
Kannada bridge	3,154	1,229	3,216	7,599

The English turn is fast enough to feel conversational, with generation and synthesis together accounting for less than a second and a half. The bridge turn is four times slower, and the difference is almost entirely provider latency rather than application work: Sarvam's REST endpoint and Deepgram's synthesis dominate. This is an acceptable trade for the bridge, which is used for vocabulary support rather than rapid repartee, but it confirms that genuinely conversational latency in every mode requires streaming rather than a full round trip.

8.3 Fluency Evaluation

Two generated samples were used to check that the fluency analyser responds to the phenomena it claims to measure. The clean sample was fluent speech at a normal rate; the hesitant sample contained filled pauses, long pauses, a repetition and a restart. Table 9 reports the results.

Table 9: Fluency analysis on two generated samples

Sample	Score	WPM	Long pauses	Fillers	Repetitions	Restarts
clean.wav	100	158.9	0	0	0	0
hesitant.wav	19	83.9	3	7	1	1

The clean sample's rate sits inside the comfortable band and produces no findings, so it scores 100. The hesitant sample's rate is below the band and each of the other three penalties is present, so the score falls to 19. The important property is not the exact number but the decomposition: the report can say that the learner spoke slowly, paused three times for more than a second, used seven fillers and restarted once, which is actionable in a way that a single holistic score is not.

8.4 Pronunciation Evaluation

Two recordings of the same sentences were analysed: one pronounced correctly, and one in which /θ/ and /ð/ were deliberately replaced by /t/ and /d/. Table 10 reports the result.

Table 10: Pronunciation analysis on correct and deliberately mispronounced speech

Recording	Score	Raw substitutions	Findings reported to the learner
Correct pronunciation	100	6	0
/θ/, /ð/ replaced by /t/, /d/	69	18	1 pattern: /θ/ → /t/, six occurrences

The table illustrates why the evidence filters exist. The acoustic model produced eighteen mismatches on the mispronounced recording and six even on the correct one; neither count is suitable for display. After notation folding, function-word handling and the recurrence, confidence and distinct-word filters, the correct recording produced no findings, and the mispronounced recording produced one claim that was both true and useful: the /θ/ sound was realised as /t/ in six occurrences across at least two words. The reduction from eighteen raw mismatches to one reported pattern is the central result of this component. It also sets the limitation: a learner who mispronounces a sound only once or twice will not be told, because the system prefers silence to a false accusation.

Warm inference ran at approximately 1.35 times real time on four CPU threads, so three minutes of speech took about two minutes to analyse. The first call additionally required 60 to 100 seconds to load the model weights, which is why the provider is held for the life of the process and why the interface warns about the first run. Memory use of about 1.6 GB is the reason the component is disabled on the free deployment.

8.5 Speech-Synthesis Comparison

Two synthesis providers were measured on one identical sentence of roughly 3.6 to 3.9 seconds of audio, requested as a complete file over REST. Table 11 reports the result.

Table 11: Time to complete audio for one sentence

Provider and voice	Time to complete audio	Role
Groq, canopylabs/orpheus-v1-english (voice troy)	about 920 ms	Primary voice in development
Deepgram, aura-2-thalia-en	about 2,780 ms	Primary voice on the free deployment; first fallback in development

Groq is roughly three times faster for a complete file, but its free-tier daily voice allowance is small, which is why the deployed configuration makes Deepgram primary and Groq the first fallback. Deepgram publishes a much lower latency figure for its streaming endpoint; that figure measures time to first byte, whereas this application requests a complete file, so the

streaming advantage does not apply. If streaming synthesis is added, the comparison should be repeated, because the ranking may reverse.

8.6 Grammar, Vocabulary and Bridge Outputs

A scripted conversation containing deliberate errors produced a grammar score of 59 out of 100 over seven sentences and eighty-three words, with four findings. The findings were a tense error (“I go to college yesterday” corrected to “I went”), an adjective used for an adverb (“explained the lesson very good” corrected to “very well”), a subject-verb agreement error (“my friend he go” corrected to “he goes”), and an article error (“a apple” corrected to “an apple”). Each carried a category, a confidence above 0.9, the quoted sentence, the correction and a one-line explanation. The transcript stored in the database was unchanged. This is the intended behaviour: the report diagnoses, and the record preserves what was actually said.

Vocabulary analysis is harder to illustrate with a single number because its value lies in the findings. Its behaviour was verified in three ways: repetition thresholds scale with conversation length; a phrase such as “very good” is reported without the preceding auxiliary; and a model that returns incorrect counts cannot change the count displayed, because counts are computed in code. Lexical diversity is reported alongside the score and is explicitly excluded from it.

The bridge was exercised with representative sentences in all four languages. In a Hindi example containing “कल मीटिंग में मेरा काम अधूरा रह गया”, the model proposed eight items; after validation and ranking the report kept “काम → work”, “थोड़ी मदद चाहिए → need a little help”, “बहुत जल्दी → very quickly”, “फ़र्क → difference”, “कल → tomorrow”, “अधूरा रह गया → remained unfinished”, “चला गया → left” and “समझ नहीं आया → didn't understand”. The loanword “मीटिंग” was never proposed, and a separate check in which the model did propose “मीटिंग में → in the meeting” and “office → workplace” confirmed that both are removed by the deterministic filters. This is the behaviour the feature was built for: the left-hand side is always the learner's language, and the right-hand side is always English.

The bridge is not perfect. In the same Hindi example, “कल” was translated as “tomorrow” although the sentence's context suggests “yesterday”; word-level translation without an explicit context instruction cannot always disambiguate. Recognition spelling noise also propagates into the report, which is why the interface carries a caveat for the languages where recognition is weakest and why the report quotes the transcript verbatim rather than a cleaned version.

8.7 Qualitative Comparison with Existing Approaches

Table 12 compares the system with the kinds of tool a learner might otherwise use. The comparison is qualitative: no controlled user study or head-to-head accuracy experiment was conducted, so no superiority claim is made.

Table 12: Qualitative comparison of feedback approaches

Approach	What it assesses	Feedback basis	Learner's own speech
Human coaching	All dimensions, holistically	Expert judgement; rater variability	Yes
Conversational AI applications	Communicative practice	Usually none, or one holistic comment	Yes, but not analysed
Pronunciation drill applications	A fixed inventory of sounds	Score against known targets	Rarely; drills are read aloud
Grammar and vocabulary checkers	Written accuracy and style	Rules or language models on text	No
Academic ASR assessment pipelines	Recognition and scoring research	Experimental, seldom deployed	Yes, in corpus settings
Lexa (this project)	Grammar, vocabulary, fluency and phoneme pronunciation	Model proposals verified by code	Yes; conversation or video

The distinctive combination is not any single analysis but the fact that all four run against one conversation, with the same transcript and the same audio, and that the results are stored and revisitable. The bridge adds a capability that none of the compared categories offers: a learner who cannot yet converse in English can still use their own language and receive English vocabulary they can reuse.

8.8 Limitations and Threats to Validity

The results must be read with several limitations in mind. The latency figures are single observations, not distributions; provider latency varies with load, region and model version. The fluency and pronunciation evaluations used generated or deliberately manipulated audio rather than recordings from a learner population, so they demonstrate that the measurements respond correctly to known phenomena, not that they correlate with human ratings. No controlled word-error-rate or character-error-rate benchmark was run for Whisper against Sarvam; the routing decision rests on the model design, the observed spelling noise and the project's own qualitative comparison, and a benchmark is stated as future work.

Language-model output varies between runs. The analyses are designed so that scoring and validation are deterministic given the model's proposals, but the set of proposed findings can change slightly. The evidence filters reduce this variability in pronunciation; they cannot eliminate it in grammar and vocabulary, where the judgement is inherently linguistic.

Several functional limitations are deliberate. Interaction is turn-based, so a learner waits for a complete reply. There is no authentication, and rate limiting is per address and in memory. Audio is not durable on the free deployment. Pronunciation analysis is unavailable there. Grammar and vocabulary can flag the same span for different reasons, and a unified report that

deduplicates across sections was planned but not implemented. The bridge's lexical lists are finite and curated, covering high-frequency vocabulary rather than dialects, slang or domain terms. Cross-session progress tracking is not available, although the schema was designed with it in mind.

Finally, the system was evaluated as an engineering artefact, not as an educational intervention. Whether learners improve with repeated use, and whether the four scores are calibrated against human ratings, are empirical questions that require a longitudinal study with human raters. The project provides the infrastructure such a study would need, and is explicit that it has not yet been conducted.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project set out to close the feedback loop around spoken-English practice: to let a learner speak freely, then show what was said, how it was said and what to work on. Lexa implements that loop end to end. A learner holds a spoken conversation with an artificial-intelligence partner in the browser; the conversation is transcribed, answered and synthesised in speech; the session is persisted; and four analyses report grammar, vocabulary, fluency and phoneme-level pronunciation against the learner's own words. An uploaded video follows the same path, and a language bridge extends the product to learners who are not yet ready to converse in English.

The engineering contribution is the discipline with which probabilistic components are contained. Language models propose findings, corrections and translations; deterministic code verifies quotations, counts occurrences, applies evidence thresholds and computes every score. The pronunciation component demonstrates why this matters: eighteen raw phoneme mismatches became one trustworthy finding, and correct speech produced none. The bridge demonstrates the same discipline at the lexical level: a proposed English-to-English “improvement” cannot survive script validation, loanword filtering and transcript matching. The result is feedback that can be explained to the learner and defended to an examiner.

The system is also engineered to run. Providers sit behind replaceable interfaces with offline implementations; synthesis degrades through an ordered fallback chain; bridge languages fall back to the default recogniser when the second vendor is unconfigured; history, uploads, analysis frequency and output length are bounded; errors share one envelope; and the deployed configuration documents exactly what it cannot do. Thirty offline test modules, with a recorded run of 454 passing tests, protect these behaviours.

The results are encouraging but bounded. English turns completed in about two seconds and bridge turns in about eight, which is usable for practice but not for real-time conversation. The fluency and pronunciation evaluations show that the measurements respond to the phenomena they claim to measure, on a small number of samples. No claim is made about learning gains or about recognition accuracy relative to a benchmark that was not run. Those are the next questions, and the architecture was built to make them answerable.

9.2 Future Scope

The following directions follow directly from the limitations above.

1. Streaming interaction. Replace the turn-based REST round trip with a persistent connection carrying incremental recognition and chunked synthesis, so that the partner begins replying before the learner's utterance has finished uploading.
2. Controlled recognition benchmarks. Evaluate Whisper and Sarvam on identical learner recordings per language, reporting word error rate, character error rate and proper-noun handling, and use the result to revisit the routing configuration.

3. Cross-session progress tracking. Use the indexed pronunciation findings and stored fluency metrics to show whether a learner's recurring errors are decreasing over weeks, and to choose what to practise next.
4. A unified report. Deduplicate findings that legitimately belong to more than one analysis, such as a wrong preposition that is both a grammatical error and an unnatural expression, and present one prioritised list.
5. Pronunciation on modest hardware. Quantise or distil the phoneme model, or move it to a scheduled job, so that pronunciation feedback can be offered on the free deployment rather than only on a larger host.
6. Durable audio and horizontal scaling. Move audio to object storage and rate-limit counters to a shared store, so that multiple instances can run behind a load balancer without losing replayable audio or double-counting requests.
7. Authentication and learner accounts. Add sign-in so that reports belong to a learner rather than to an opaque session identifier, which is a prerequisite for progress tracking.
8. Broader language coverage. Add languages to the bridge by supplying script metadata and lexical lists, and evaluate Sarvam's code-mixed output mode for Hinglish and comparable speech.
9. Calibration against human ratings. Conduct a study in which teachers rate the same recordings, and compare the four scores with those ratings; recalibrate thresholds where the agreement is weak.
10. A mobile client. Provide a native capture path where browser recording is unreliable, reusing the same API.

The project therefore ends with a working system and a clear agenda. Its central lesson is that a useful speech coach is not produced by any single model. It is produced by arranging several imperfect models around a durable record of what the learner said, and by keeping the judgement that matters in code that can be tested.

REFERENCES

[1] Council of Europe, Common European Framework of Reference for Languages: Learning, Teaching, Assessment. Cambridge, U.K.: Cambridge Univ. Press, 2001.

[2] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 12449–12460.

[3] A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," in Proc. Interspeech, 2021, pp. 2426–2430.

[4] R. Ardila et al., "Common Voice: A massively-multilingual speech corpus," in Proc. Language Resources and Evaluation Conf. (LREC), 2020, pp. 4218–4222.

[5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. Int. Conf. Mach. Learn. (ICML), 2023, pp. 28492–28518.

[6] A. Vaswani et al., "Attention is all you need," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 5998–6008.

[7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, San Francisco, CA, USA, Tech. Rep., 2018.

[8] T. B. Brown et al., "Language models are few-shot learners," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 1877–1901.

[9] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," arXiv:2508.10925, 2025.

[10] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," J. Mol. Biol., vol. 48, no. 3, pp. 443–453, 1970.

[11] Carnegie Mellon University, "The CMU Pronouncing Dictionary," 2015. [Online]. Available: http://www.speech.cs.cmu.edu/cgi-bin/cmudict

[12] K. Park, "g2p_en: A Python package for grapheme-to-phoneme conversion," GitHub, 2019. [Online]. Available: https://github.com/Kyubyong/g2p

[13] H. H. Clark and J. E. Fox Tree, "Using uh and um in spontaneous speaking," Cognition, vol. 84, no. 1, pp. 73–111, 2002.

[14] Unicode Consortium, The Unicode Standard, Version 15.0.0. Mountain View, CA, USA: Unicode, 2022.

[15] Mozilla, "MediaRecorder API," MDN Web Docs, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder

[16] Silero Team, "Silero VAD: pre-trained enterprise-grade voice activity detector," GitHub, 2026. [Online]. Available: https://github.com/snakers4/silero-vad

[17] FFmpeg Project, "FFmpeg documentation," 2026. [Online]. Available: https://ffmpeg.org/documentation.html

[18] Sarvam AI, "Speech to Text API," Sarvam Documentation, 2026. [Online]. Available: https://docs.sarvam.ai/api-reference-docs/speech-to-text/transcribe

[19] Groq, "Speech to text," Groq Documentation, 2026. [Online]. Available: https://console.groq.com/docs/speech-to-text

[20] Groq, "API reference: chat completions and speech synthesis," Groq Documentation, 2026. [Online]. Available: https://console.groq.com/docs/api-reference

[21] Deepgram, "Text-to-speech models," Deepgram Documentation, 2026. [Online]. Available: https://developers.deepgram.com/docs/tts-models

[22] S. Ramírez, FastAPI Documentation, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[23] SQLAlchemy, "SQLAlchemy 2.0 documentation," 2026. [Online]. Available: https://docs.sqlalchemy.org/

[24] Meta Open Source, "React documentation," 2026. [Online]. Available: https://react.dev/

[25] TanStack, "TanStack Router and TanStack Start documentation," 2026. [Online]. Available: https://tanstack.com/router

[26] FFmpeg Project, "FFprobe documentation," 2026. [Online]. Available: https://ffmpeg.org/ffprobe.html

[27] Render, "Deploy for free," Render Documentation, 2026. [Online]. Available: https://render.com/docs/free

[28] Neon, "Neon documentation," 2026. [Online]. Available: https://neon.tech/docs

[29] Vercel, "Vercel documentation," 2026. [Online]. Available: https://vercel.com/docs